

SECTION 8

INSTRUCTION EXECUTION TIMING

This section describes the instruction execution and operations (table searches, etc.) of the MC68020/EC020 in terms of external clock cycles. It provides accurate execution and operation timing guidelines but not exact timings for every possible circumstance. This approach is used since exact execution time for an instruction or operation is highly dependent on memory speeds and other variables. The timing numbers presented in this section allow the assembly language programmer or compiler writer to predict timings needed to evaluate the performance of the MC68020/EC020.

In this section, instruction and operation times are shown in clock cycles, which eliminates clock frequency dependencies.

8.1 TIMING ESTIMATION FACTORS

The advanced architecture of the MC68020/EC020 makes exact instruction timing calculations difficult due to the effects of:

1. An On-Chip Instruction Cache and Instruction Prefetch
2. Operand Misalignment
3. Bus Controller/Sequence Concurrency
4. Instruction Execution Overlap

These factors make MC68020/EC020 instruction set timing difficult to calculate on a single instruction basis since instructions vary in execution time from one context to another. A detailed explanation of each of these factors follows.

8.1.1 Instruction Cache and Prefetch

The on-chip cache of the MC68020/EC020 is an instruction-only cache. Its purpose is to increase execution efficiency by providing a quick store for instructions.

Instruction prefetches that hit in the cache will occur with no delay in instruction execution. Instruction prefetches that miss in the cache will cause an external memory cycle to be performed, which may overlap with internal instruction execution. Thus, while the execution unit of the microprocessor is busy, the bus controller prefetches the next instruction from external memory. Both cases are illustrated in later examples.

When prefetching instructions from external memory, the microprocessor will utilize long-word read cycles. When the read is aligned on a long-word address boundary, the processor reads two words, which may load two instructions at once or two words of a multiword instruction. The subsequent instruction prefetch will find the second word is already available, and there is no need to run an external bus cycle (read).

The MC68020/EC020 always prefetches long words. When an instruction prefetch falls on an odd-word boundary (e.g., due to a branch to an odd-word location), the MC68020/EC020 will read the even word associated with the long-word base address at the same time as (32-bit memory) or before (8- or 16-bit memory) the odd word is read. When an instruction prefetch falls on an even-word boundary (as would be the normal case), the MC68020/EC020 reads both words at the long-word address, thus effectively prefetching the next two words.

8.1.2 Operand Misalignment

Another significant factor affecting instruction timing is operand misalignment. Operand misalignment has impact on performance when the microprocessor is reading or writing external memory. In this case, the address of a word operand falls across a long-word boundary, or a long-word operand falls on a byte or word address that is not a long-word boundary. Although the MC68020/EC020 will automatically handle all occurrences of operand misalignment, it must use multiple bus cycles to complete such transfers.

8.1.3 Bus/Sequencer Concurrency

The bus controller is responsible for all bus activity. The sequencer controls the bus controller, instruction execution, and internal processor operation, such as calculation of effective addresses and setting of condition codes.

The bus controller and sequencer can operate on an instruction concurrently. The bus controller can perform a read or write while the sequencer controls an effective address calculation or sets the condition codes. The sequencer may also request a bus cycle that the bus controller cannot immediately perform. In this case, the bus cycle is queued and the bus controller runs the cycle when the current cycle is complete.

8.1.4 Instruction Execution Overlap

Overlap is the time, measured in clocks, when two instructions execute concurrently. In Figure 8-1, instructions A and B execute concurrently, and the overlapped portion of instruction B is absorbed in the instruction execution time of A (the previous instruction). The overlap time is deducted from the execution time of instruction B. Similarly, there is an overlap period between instruction B and instruction C, which reduces the attributed execution time for C.

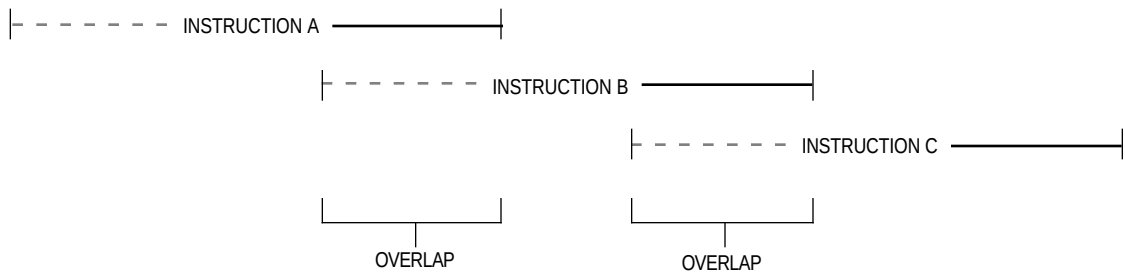


Figure 8-1. Concurrent Instruction Execution

The execution time attributed to instructions A, B, and C (after considering the overlap) is depicted in Figure 8-2.

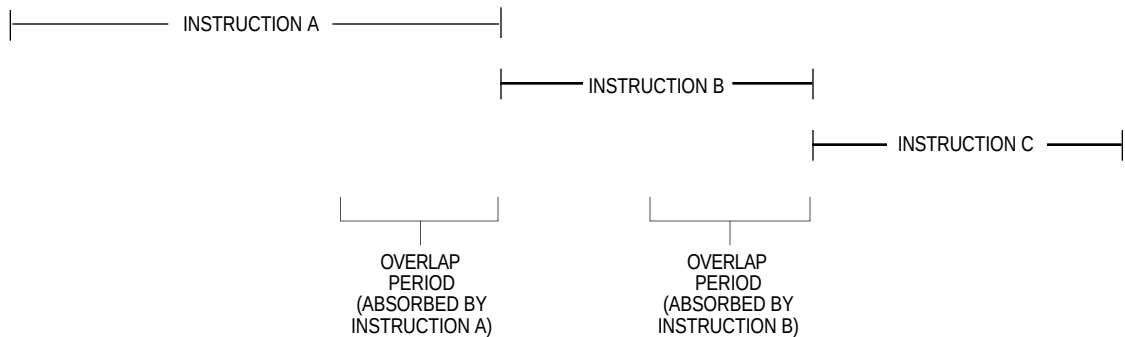


Figure 8-2. Instruction Execution for Instruction Timing Purposes

It is possible that the execution time of an instruction will be absorbed by the overlap with a previous instruction for a net execution time of zero clocks.

Because of this overlap, a NOP is required between a write to a peripheral to clear an interrupt request and a subsequent MOVE to SR instruction to lower the interrupt mask level. Otherwise, the MOVE to SR instruction may complete before the write is accomplished, and a new interrupt exception will be generated for an old interrupt request.

8.1.5 Instruction Stream Timing Examples

A programming example allows a more detailed examination of these effects. The effect of instruction execution overlap on instruction timing is illustrated by the following example instruction stream.

Instruction	
#1)	MOVE.L D4,(A1)+
#2)	ADD.L D4,D5
#3)	MOVE.L (A1), -(A2)
#4)	ADD.L D5,D6

Example 1

For the first example, the assumptions are:

- 1. The data bus is 32 bits,
- 2. The first instruction is prefetched from an odd-word address,
- 3. Memory access occurs with no wait states, and
- 4. The instruction cache is disabled.

For example 1, the instruction stream is positioned in 32-bit memory as follows:

Address	n	...	MOVE #1
	n + 4	ADD #2	MOVE #3
	n + 8	ADD #4	...

Figure 8-3 shows processor activity on the first example instruction stream. It shows the activity of the external bus, the bus controller, the sequencer, and the attributed instruction execution time.

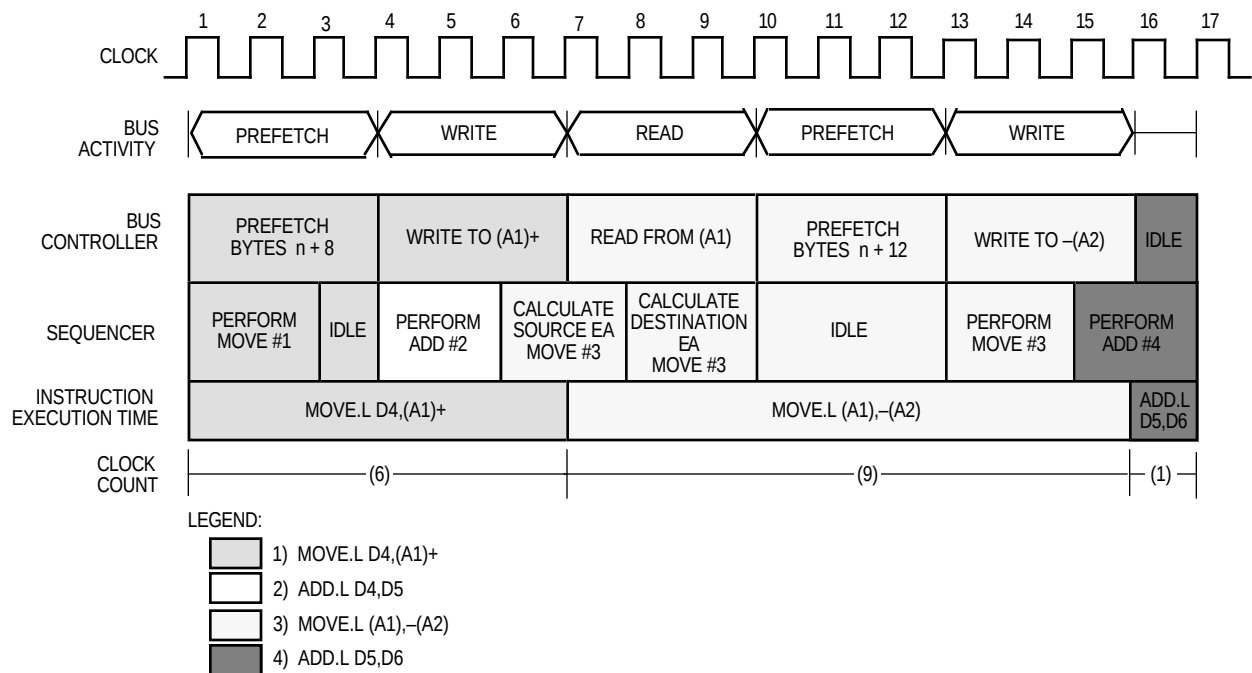


Figure 8-3. Processor Activity for Example 1

For the first three clocks of this example, the bus controller and sequencer are both performing tasks associated with the MOVE #1 instruction. The next three clocks (clocks 4, 5, and 6) demonstrate instruction overlap. The bus controller is performing a write to memory as part of the MOVE #1 instruction. The sequencer, on the other hand, is performing the ADD #2 instruction for two clocks (clocks 4 and 5) and beginning source effective address (EA) calculations for the MOVE #3 instruction. The bus controller activity completely overlaps the execution of the ADD #2 instruction, causing the ADD #2 attributed execution time to be zero clocks. The overlap also shortens the effective execution time of the MOVE #3 instruction by one clock because the bus controller completes the MOVE #1 write operation while the sequencer begins the MOVE #3 EA calculation.

The sequencer continues the source EA calculation for one more clock period (clock 7) while the bus controller begins a read for MOVE #3. When counting instruction execution time in bus clocks, the MOVE #1 completes at the end of clock 6, and the execution of MOVE #3 begins on clock 7.

Both the sequencer and bus controller continue with MOVE #3 until the end of clock 14, when the sequencer begins to perform ADD #4. Timing for MOVE #3 continues because the bus controller is still performing the write to the destination of MOVE #3. The bus activity for MOVE #3 completes at the end of clock 15. The effective execution time for MOVE #3 is nine clocks.

The one clock cycle (clock 15) when the sequencer is performing ADD #4 and the bus controller is writing to the destination of MOVE #3 is absorbed by the execution time of MOVE #3. This overlap shortens the effective execution time of ADD #4 by one clock, giving it an attributed execution time of one clock.

Example 2

Using the same instruction stream, the second example demonstrates the different effects of instruction execution overlap on instruction timing when the same instructions are positioned slightly differently in 32-bit memory:

Address	n	MOVE #1	ADD #2
	n + 4	MOVE #3	ADD #4
	n + 8	...	...

The assumptions for example 2 (see Figure 8-4) are:

1. The data bus is 32 bits,
2. The first instruction is prefetched from an even-word address,
3. Memory access occurs with no wait states, and
4. The cache is disabled.

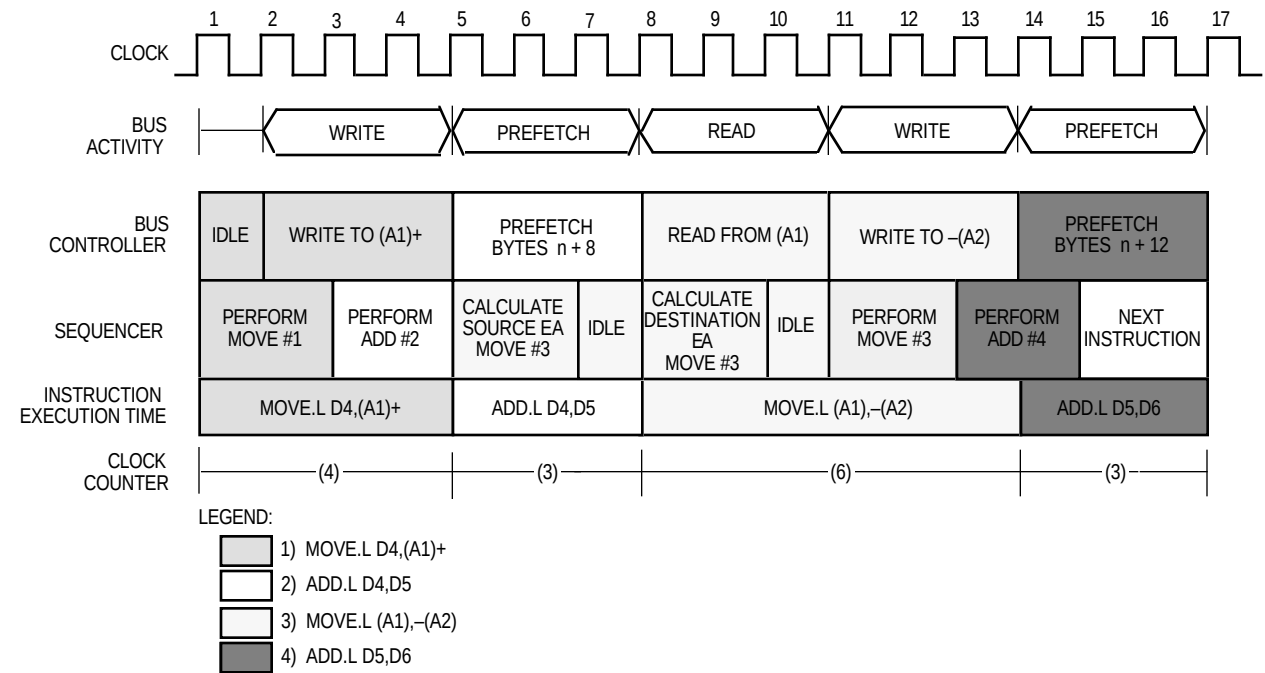


Figure 8-4. Processor Activity for Example 2

Although the total execution time of the instruction segment does not change in this example, the individual instruction times are significantly different. This example demonstrates that the effects of overlap are not only instruction-sequence dependent but are also dependent upon the alignment of the instruction stream in memory.

Example 3

Both Figures 8-3 and 8-4 show instruction execution without benefit of the MC68020/EC020 instruction cache. Figure 8-5 shows a third example for the same instruction stream executing in the cache. Note that once the instructions are in the cache, the original location in external memory is no longer a factor in timing.

The assumptions for Example 3 are:

1. The data bus is 32 bits,
2. The cache is enabled and instructions are in the cache, and
3. Memory access occurs with no wait states.

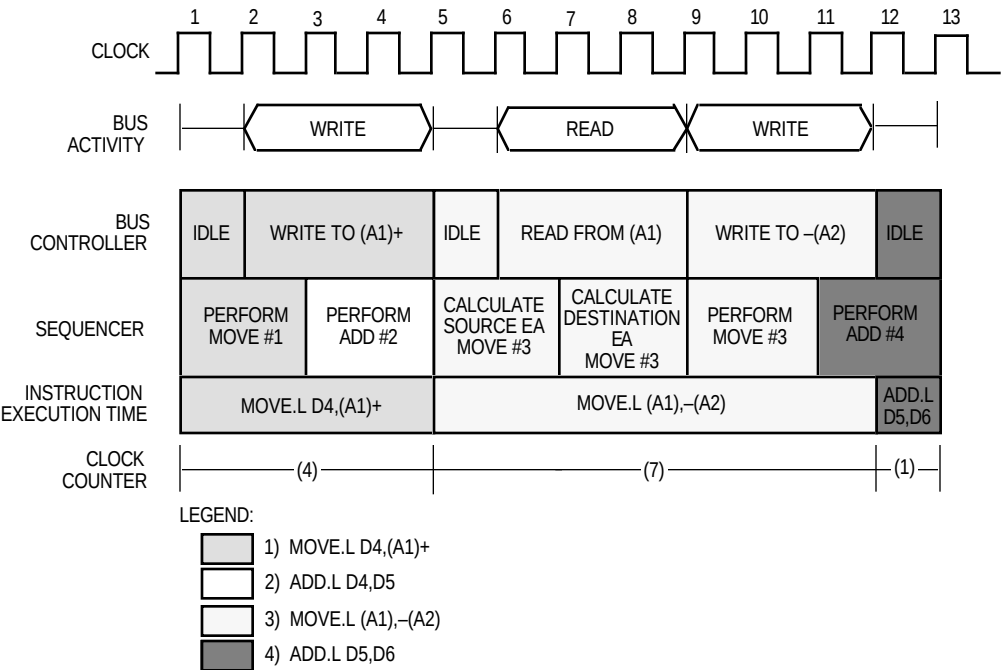


Figure 8-5. Processor Activity for Example 3

Figure 8-5 illustrates the benefits of the instruction cache. The total number of clock cycles is reduced from 16 to 12 clocks. Since the instructions are resident in the cache, the instruction prefetch activity does not require the bus controller to perform external bus cycles. Since prefetch occurs with no delay, the bus controller is idle more often.

Example 4

Idle clock cycles, such as those shown in example 3, are useful in MC68020/EC020 systems that require wait states when accessing external memory. This fact is illustrated in example 4 (see Figure 8-6) with the following assumptions:

1. The data bus is 32 bits,
2. The cache is enabled and instructions are in the cache, and
3. Memory access occurs with one wait state.

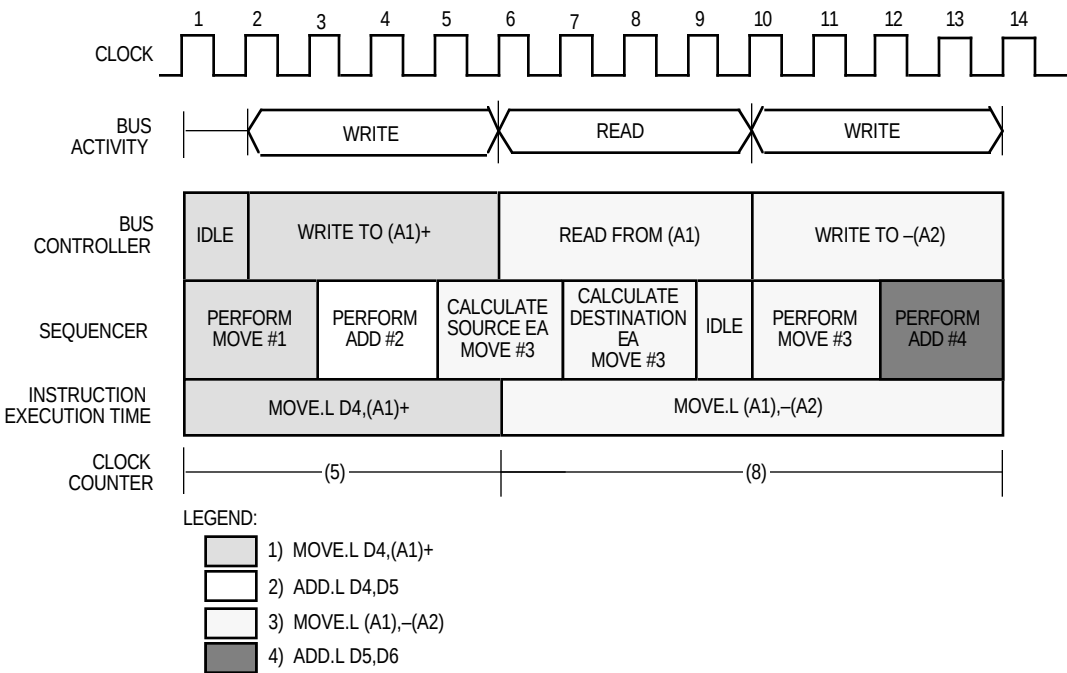


Figure 8-6. Processor Activity for Example 4

Figure 8-6 shows the same instruction stream executing with four clocks for every read and write. The idle bus cycles coincide with the wait states of the memory access; therefore, the total execution time is only 13 clocks.

Examples 1–4 demonstrate the complexity of instruction timing calculation for the MC68020/EC020. It is impossible to anticipate individual instruction timing as an absolute number of clock cycles due to the dependency of overlap on the instruction sequence and alignment as well as the number of wait states in memory. This can be seen by comparing individual and composite time for Figures 8-3 through 8-6. These instruction timings are compared in Table 8-1, where timing varies for each instruction as the context varies.

Table 8-1. Examples 1–4 Instruction Stream Execution Comparison

Instruction	Example 1 (Odd Alignment)	Example 2 (Even Alignment)	Example 3 (Cache)	Example 4 (Cache with Wait States)
#1) MOVE.L D4,(A1)+	6	4	4	5
#2) ADD.L D4,D5	0	3	0	0
#3) MOVE.L (A1),-(A2)	9	6	7	8
#4) ADD.L D5,D6	1	3	1	0
Total Clock Cycles	16	16	12	13

8.2 INSTRUCTION TIMING TABLES

The instruction times given in the following illustration include the following assumptions about the MC68020/EC020 system:

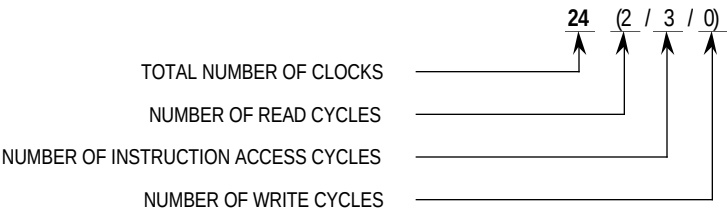
- 1. All operands are long-word aligned as is the stack,
- 2. The data bus is 32 bits, and
- 3. Memory access occurs with no wait states (three-cycle read/write).

There are three values given for each instruction and addressing mode:

- 1. The best case (BC), which reflects the time (in clocks) when the instruction is in the cache and benefits from maximum overlap due to other instructions,
- 2. The cache-only case (CC) when the instruction is in the cache but has no overlap, and
- 3. The worst case (WC) when the instruction is not in the cache or the cache is disabled and there is no instruction overlap.

The only instances for which the size of the operand has any effect are the instructions with immediate operands. Unless specified otherwise, immediate byte and word operands have identical execution times.

Within each set or column of instruction timings are four sets of numbers, three of which are enclosed in parentheses. The bolded outer number is the total number of clocks for the instruction. The first number inside the parentheses is the number of operand read cycles performed by the instruction. The second value inside parentheses is the number of instruction accesses performed by the instruction, including all prefetches to keep the instruction pipe filled. The third value within parentheses is the number of write cycles performed by the instruction. One example from the instruction timing table is:



The total number of bus-activity clocks for the previous example is derived in the following way:

$$\begin{aligned} &(2 \text{ Reads} * 3 \text{ Clocks/Read}) + (3 \text{ Instruction Accesses} * 3 \text{ Clocks/Access}) \\ &\quad + (0 \text{ Writes} * 3 \text{ Clocks/Write}) = 15 \text{ Clocks of Bus Activity} \\ &24 \text{ Total Clocks} - 15 \text{ Clocks (Bus Activity)} = 9 \text{ Internal Clocks} \end{aligned}$$

The example used here was taken from a worst-case fetch effective address time. The addressing mode was ([d₃₂,B],l,d₃₂). The same addressing mode under the best-case

entry is 17 (2/0/0). For the best case, there are no instruction accesses because the cache is enabled and the sequencer does not have to go to external memory for the instruction words.

The first tables deal exclusively with fetching and calculating effective addresses and immediate operands. The tables are arranged in this manner because some instructions do not require effective address calculation or fetching. For example, the instruction CLR <ea> (found in the table under **8.2.11 Single Operand Instructions**) only needs to have a calculated effective address time added to its table entry because no fetch of an operand is required. This instruction only writes to memory or a register. Some instructions use specific addressing modes which exclude timing for calculation or fetching of an operand. When these instances arise, they are footnoted to indicate which other tables are needed in the timing calculation.

Many two-word instructions (e.g., MULU.L, DIV.L, BFSET, etc.) include the fetch immediate effective address time or the calculate immediate effective address time in the execution time calculation. The timing for immediate data of word length (#<data>.W) is used for these calculations. If the instruction has a source and a destination, the source effective address is used for the table lookup. If the instruction is single operand, the effective address of that operand is used.

The following example includes multiword instructions that refer to the fetch immediate effective address and calculate immediate effective address tables in **8.2 Instruction Timing Tables**.

Instruction		
#1) MULU.L	D7,D1:D2	
#2) BFCLR	\$6000{0:8}	
#3) DIVS.L	#\$10000,D3:D4	
		CC
1.MULU.L (D7),D1:D2		
#<data>.W,Dn		2
MUL.L EA,Dn		43
2.BFCLR \$6000{0:8}		
#<data>.W,,\$XXX.W		5
BFCLR Mem (<5 bytes)		16
3.DIVS.L #\$10000,D3:D4		
#<data>.W,#<data>.L		6
DIVS.L EA, Dn		90
Execution time		= 2 + 43 + 5 + 16 + 6 + 90
		= 102 clock periods

NOTE

This CC time is a maximum since the times given for the MULU.L and DIVS.L are maximums.

The MOVE instruction timing tables include all necessary timing for extension word fetch, address calculation, and operand fetch.

The instruction timing tables are used to calculate a best-case and worst-case bounds for some target instruction stream. Calculating exact timing from the timing tables is impossible because the tables cannot anticipate how the combination of factors will influence every particular sequence of instructions. This is illustrated by comparing the observed instruction timing from the prior four examples with instruction timing derived from the instruction timing tables.

Table 8-2 lists the original instruction stream and the corresponding clock timing from the appropriate timing tables for the best case, cache-only case, and worst case.

Table 8-2. Instruction Timings from Timing Tables

Instruction		Best Case	Cache Case	Worst Case
#1) MOVE.L	D4,(A1)+	4	4	6
#2) ADD.L	D4,D5	0	2	3
#3) MOVE.L	(A1),-(A2)	6	7	9
#4) ADD.L	D5,D6	0	2	3
Total		10	15	21

Table 8-3 summarizes the observed instruction timings for the same instruction stream as executed according to the assumptions of the four examples. For each example, Table 8-3 shows which entry (BC/CC/WC) from the timing tables corresponds to the observed timing for each of the four instructions. Some of the observed instruction timings cannot be found in the timing tables and appear in Table 8-3 within parentheses in the most appropriate column. These timings occur when instruction execution overlap dynamically alters what would otherwise be a BC, CC, or WC timing.

Table 8-3. Observed Instruction Timings

Instruction		Example 1			Example 2			Example 3			Example 4		
		BC	CC	WC	BC	CC	WC	BC	CC	WC	BC	CC	WC
#1) MOVE.L	D4,(A1)+			6	4				4			(5)	
#2) ADD.L	D4,D5	0					3	0			0		
#3) MOVE.L	(A1),-(A2)			9	6				7			(8)	
#4) ADD.L	D5,D6	(1)					3	(1)			0		
Total			(16)			(16)			(12)			(13)	

Comparing Tables 8-2 and 8-3 demonstrates that calculation of instruction timing cannot be a simple lookup of only BC or only WC timings. Even when the assumptions are known and fixed, as in the four examples summarized in Table 8-3, the microprocessor can sometimes achieve best-case timings under worst-case assumptions.

Looking across the four examples in Table 8-3 for an individual instruction, it is difficult to predict which timing table entry is used, since the influence of instruction overlap may or may not improve the BC, WC, or CC timings. When looking at the observed instruction timings for one example, it is also difficult to determine which combination of BC/CC/WC timing is required. Just how the instruction stream will fit and run with the cache enabled, how instructions are positioned in memory, and the degree of instruction overlap are factors that are impossible to account for in all combinations of the timing tables.

Although the timing tables cannot accurately predict the instruction timing that would be observed when executing an instruction stream on the MC68020/EC020, the tables can be used to calculate best-case and worst-case bounds for instruction timing. Absolute instruction timing must be measured by using the microprocessor itself to execute the target instruction stream.

8.2.1 Fetch Effective Address

The fetch effective address table indicates the number of clock periods needed for the processor to calculate and fetch the specified effective address. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Address Mode	Best Case	Cache Case	Worst Case
Dn	0(0/0/0)	0(0/0/0)	0(0/0/0)
An	0(0/0/0)	0(0/0/0)	0(0/0/0)
(An)	3(1/0/0)	4(1/0/0)	4(1/0/0)
(An)+	4(1/0/0)	4(1/0/0)	4(1/0/0)
-(An)	3(1/0/0)	5(1/0/0)	5(1/0/0)
(d ₁₆ ,An) of (d ₁₆ ,PC)	3(1/0/0)	5(1/0/0)	6(1/1/0)
(xxx).W	3(1/0/0)	4(1/0/0)	6(1/1/0)
(xxx).L	3(1/0/0)	4(1/0/0)	7(1/1/0)
#<data>.B	0(0/0/0)	2(0/0/0)	3(0/1/0)
#<data>.W	0(0/0/0)	2(0/0/0)	3(0/1/0)
#<data>.L	0(0/0/0)	4(0/0/0)	5(0/1/0)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	4(1/0/0)	7(1/0/0)	8(1/1/0)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	4(1/0/0)	7(1/0/0)	9(1/1/0)
(B)	4(1/0/0)	7(1/0/0)	9(1/1/0)
(d ₁₆ ,B)	6(1/0/0)	9(1/0/0)	12(1/1/0)
(d ₃₂ ,B)	10(1/0/0)	13(1/0/0)	16(1/2/0)
([B],I)	9(2/0/0)	12(2/0/0)	13(2/1/0)
([B],I,d ₁₆)	11(2/0/0)	14(2/0/0)	16(2/1/0)
([B],I,d ₃₂)	11(2/0/0)	14(2/0/0)	17(2/2/0)
([d ₁₆ ,B],I)	11(2/0/0)	14(2/0/0)	16(2/1/0)
([d ₁₆ ,B],I,d ₁₆)	13(2/0/0)	16(2/0/0)	19(2/2/0)
([d ₁₆ ,B],I,d ₃₂)	13(2/0/0)	16(2/0/0)	20(2/2/0)
([d ₃₂ ,B],I)	15(2/0/0)	18(2/0/0)	20(2/2/0)
([d ₃₂ ,B],I,d ₁₆)	17(2/0/0)	20(2/0/0)	22(2/2/0)
([d ₃₂ ,B],I,d ₃₂)	17(2/0/0)	20(2/0/0)	24(2/3/0)

B = Base address; 0, An, PC, Xn, An + Xn. Form does not affect timing.

I = Index; 0, Xn

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

8.2.2 Fetch Immediate Effective Address

The fetch immediate effective address table indicates the number of clock periods needed for the processor to fetch the immediate source operand and calculate and fetch the specified destination operand. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Address Mode	Best Case	Cache Case	Worst Case
#<data>.W,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
#<data>.L,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
#<data>.W,(An)	3(1/0/0)	4(1/0/0)	4(1/1/0)
#<data>.L,(An)	3(1/0/0)	4(1/0/0)	7(1/1/0)
#<data>.W,(An)+	4(1/0/0)	6(1/0/0)	7(1/1/0)
#<data>.L,(An)+	5(1/0/0)	8(1/0/0)	9(1/1/0)
#<data>.W,—(An)	3(1/0/0)	5(1/0/0)	6(1/1/0)
#<data>.L,—(An)	4(1/0/0)	7(1/0/0)	8(1/1/0)
#<data>.W,(bd,An)	3(1/0/0)	5(1/0/0)	7(1/1/0)
#<data>.L,(bd,An)	4(1/0/0)	7(1/0/0)	10(1/2/0)
#<data>.W,xxx.W	3(1/0/0)	5(1/0/0)	7(1/1/0)
#<data>.L,xxx.W	4(1/0/0)	7(1/0/0)	10(1/2/0)
#<data>.W,xxx.L	3(1/0/0)	6(1/0/0)	10(1/2/0)
#<data>.L,xxx.L	4(1/0/0)	8(1/0/0)	12(1/2/0)
#<data>.W,#<data>.B,W	0(0/0/0)	4(0/0/0)	6(0/2/0)
#<data>.L,#<data>.B,W	1(0/0/0)	6(0/0/0)	8(0/2/0)
#<data>.W,#<data>.L	0(0/0/0)	6(0/0/0)	8(0/2/0)
#<data>.L,#(data>.L	1(0/0/0)	8(0/0/0)	10(0/2/0)
#<data>.W,(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	4(1/0/0)	9(1/0/0)	11(1/2/0)
#<data>.L,(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	5(1/0/0)	11(1/0/0)	13(1/2/0)
#<data>.W,(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	4(1/0/0)	9(1/0/0)	12(1/2/0)
#<data>.L,(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	5(1/0/0)	11(1/0/0)	15(1/2/0)
#<data>.W,(B)	4(1/0/0)	9(1/0/0)	12(1/1/0)
#<data>.L,(B)	5(1/0/0)	11(1/0/0)	14(1/2/0)
#<data>.W,(bd,PC)	10(1/0/0)	15(1/0/0)	19(1/3/0)
#<data>.L,(bd,PC)	11(1/0/0)	17(1/0/0)	21(1/3/0)
#<data>.W,(d ₁₆ ,B)	6(1/0/0)	11(1/0/0)	15(1/2/0)
#<data>.L,(d ₁₆ ,B)	7(1/0/0)	13(1/0/0)	17(1/2/0)
#<data>.W,(d ₃₂ ,B)	10(1/0/0)	15(1/0/0)	19(1/3/0)
#<data>.L,(d ₃₂ ,B)	11(1/0/0)	17(1/0/0)	21(1/3/0)
#<data>.W,([B],l)	9(2/0/0)	14(2/0/0)	16(2/2/0)
#<data>.L,([B],l)	10(2/0/0)	16(2/0/0)	18(2/2/0)
#<data>.W,([B],l,d ₁₆)	11(2/0/0)	16(2/0/0)	19(2/2/0)
#<data>.L,([B],l,d ₁₆)	12(2/0/0)	18(2/0/0)	21(2/2/0)

Address Mode	Best Case	Cache Case	Worst Case
#<data>.W,([B],I,d ₃₂)	11(2/0/0)	16(2/0/0)	20(2/2/0)
#<data>.L,([d ₁₆ ,B],I,d ₃₂)	12(2/0/0)	18(2/0/0)	22(2/3/0)
#<data>.W,([d ₁₆ ,B],I)	11(2/0/0)	16(2/0/0)	19(2/2/0)
#<data>.L,([d ₁₆ ,B],I)	12(2/0/0)	18(2/0/0)	21(2/2/0)
#<data>.W,([d ₁₆ ,B],I,d ₁₆)	13(2/0/0)	18(2/0/0)	22(2/2/0)
#<data>.L,([d ₁₆ ,B],I,d ₁₆)	14(2/0/0)	20(2/0/0)	24(2/3/0)
#<data>.W,([d ₃₂ ,B],I)	15(2/0/0)	20(2/0/0)	23(2/3/0)
#<data>.L,([d ₃₂ ,B],I)	16(2/0/0)	22(2/0/0)	25(2/3/0)
#<data>.W,([d ₃₂ ,B],I,d ₁₆)	17(2/0/0)	22(2/0/0)	25(2/3/0)
#<data>.L,([d ₃₂ ,B],I,d ₁₆)	18(2/0/0)	24(2/0/0)	27(2/3/0)
#<data>.W,([d ₃₂ ,b],I,d ₃₂)	17(2/0/0)	22(2/0/0)	27(2/3/0)
#<data>.L,([d ₃₂ ,b],I,d ₃₂)	18(2/0/0)	24(2/0/0)	29(2/4/0)

B = Base address; 0, An, PC, Xn, An + Xn. Form does not affect timing.

I = Index; 0, Xn

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

8.2.3 Calculate Effective Address

The calculate immediate effective address table indicates the number of clock periods needed for the processor to calculate the specified effective address. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Address Mode	Best Case	Cache Case	Worst Case
Dn	0(0/0/0)	0(0/0/0)	0(0/0/0)
An	0(0/0/0)	0(0/0/0)	0(0/0/0)
(An)	2(0/0/0)	2(0/0/0)	2(0/0/0)
(An)+	2(0/0/0)	2(0/0/0)	2(0/0/0)
-(An)	2(0/0/0)	2(0/0/0)	2(0/0/0)
(d ₁₆ ,An) or (d ₁₆ ,PC)	2(0/0/0)	2(0/0/0)	3(0/1/0)
<data>.W	2(0/0/0)	2(0/0/0)	3(0/1/0)
<data>.L	1(0/0/0)	4(0/0/0)	5(0/1/0)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	1(0/0/0)	4(0/0/0)	5(0/1/0)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	3(0/0/0)	6(0/0/0)	7(0/1/0)
(B)	3(0/0/0)	6(0/0/0)	7(0/1/0)
(d ₁₆ ,B)	5(0/0/0)	8(0/0/0)	10(0/1/0)
(d ₃₂ ,B)	9(0/0/0)	12(0/0/0)	15(0/2/0)
([B],I)	8(1/0/0)	11(1/0/0)	12(1/1/0)
([B],I,d ₁₆)	10(1/0/0)	13(1/0/0)	15(1/1/0)
([B],I,d ₃₂)	10(1/0/0)	13(1/0/0)	16(1/2/0)
([d ₁₆ ,B],I)	10(1/0/0)	13(1/0/0)	15(1/1/0)
([d ₁₆ ,B],I,d ₁₆)	12(1/0/0)	15(1/0/0)	18(1/2/0)
([d ₁₆ ,B],I,d ₃₂)	12(1/0/0)	15(1/0/0)	19(1/2/0)
([d ₃₂ ,B],I)	14(1/0/0)	17(1/0/0)	19(1/2/0)
([d ₃₂ ,B],I,d ₁₆)	16(1/0/0)	19(1/0/0)	21(1/2/0)
([d ₃₂ ,B],I,d ₃₂)	16(1/0/0)	19(1/0/0)	24(1/3/0)

B = Base address; 0, An, PC, Xn, An + Xn. Form does not affect timing.

I = Index; 0, Xn

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

8.2.4 Calculate Immediate Effective Address

The calculate immediate effective address table indicates the number of clock periods needed for the processor to fetch the immediate source operand and calculate the specified destination effective address. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Address Mode	Best Case	Cache Case	Worst Case
#<data>.W,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
#<data>.L,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
#<data>.W,(An)	0(0/0/0)	2(0/0/0)	3(0/1/0)
#<data>.L,(An)	1(0/0/0)	4(0/0/0)	5(0/1/0)
#<data>.W,(An)+	2(0/0/0)	4(0/0/0)	5(0/1/0)
#<data>.L,(An)+	3(0/0/0)	6(0/0/0)	7(0/1/0)
#<data>.W,(bd,An)	1(0/0/0)	4(0/0/0)	5(0/1/0)
#<data>.L,(bd,An)	3(0/0/0)	6(0/0/0)	8(0/2/0)
#<data>.W,xxx.W	1(0/0/0)	4(0/0/0)	5(0/1/0)
#<data>.L,xxx.W	3(0/0/0)	6(0/0/0)	8(0/2/0)
#<data>.W,xxx.L	2(0/0/0)	4(0/0/0)	6(0/2/0)
#<data>.L,xxx.L	3(0/0/0)	8(0/0/0)	10(0/2/0)
#<data>.W,(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	0(0/0/0)	6(0/0/0)	8(0/2/0)
#<data>.L,(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	2(0/0/0)	8(0/0/0)	10(0/2/0)
#<data>.W,(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	3(0/0/0)	8(0/0/0)	10(0/2/0)
#<data>.L,(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	4(0/0/0)	10(0/0/0)	12(0/2/0)
#<data>.W,(B)	3(0/0/0)	8(0/0/0)	10(0/1/0)
#<data>.L,(B)	4(0/0/0)	10(0/0/0)	12(0/2/0)
#<data>.W,(bd,PC)	9(0/0/0)	14(0/0/0)	18(0/3/0)
#<data>.L,(bd,PC)	10(0/0/0)	16(0/0/0)	20(0/3/0)
#<data>.W,(d ₁₆ ,B)	5(0/0/0)	10(0/0/0)	13(0/2/0)
#<data>.L,(d ₁₆ ,B)	6(0/0/0)	12(0/0/0)	15(0/2/0)
#<data>.W,(d ₃₂ ,B)	9(0/0/0)	14(0/0/0)	18(0/2/0)
#<data>.L,(d ₃₂ ,B)	10(0/0/0)	16(0/0/0)	20(0/3/0)
#<data>.W,([B],l)	8(1/0/0)	13(1/0/0)	15(1/2/0)
#<data>.L,([B],l)	9(1/0/0)	15(1/0/0)	17(1/2/0)
#<data>.W,([B],l,d ₁₆)	10(1/0/0)	15(1/0/0)	18(1/2/0)
#<data>.L,([B],l,d ₁₆)	11(1/0/0)	17(1/0/0)	20(1/2/0)
#<data>.W,([B],l,d ₃₂)	10(1/0/0)	15(1/0/0)	19(1/2/0)
#<data>.L,([d ₁₆ ,B],l,d ₃₂)	11(1/0/0)	17(1/0/0)	21(1/3/0)
#<data>.W,([d ₁₆ ,B],l)	10(1/0/0)	15(1/0/0)	18(1/2/0)
#<data>.L,([d ₁₆ ,B],l)	11(1/0/0)	17(1/0/0)	20(1/2/0)
#<data>.W,([d ₁₆ ,B],l,d ₁₆)	12(1/0/0)	17(1/0/0)	21(1/2/0)

Address Mode	Best Case	Cache Case	Worst Case
#<data>.L,([d ₁₆ ,B],I,d ₁₆)	13(1/0/0)	19(1/0/0)	23(1/3/0)
#<data>.[d ₁₆ ,B],I,d ₃₂)	12(1/0/0)	17(1/0/0)	22(1/3/0)
#<data>.[d ₁₆ ,B],I,d ₃₂)	13(1/0/0)	19(1/0/0)	24(1/3/0)
#<data>.W,([d ₃₂ ,B],I)	14(1/0/0)	19(1/0/0)	22(1/3/0)
#<data>.L,([d ₃₂ ,B],I)	15(1/0/0)	21(1/0/0)	24(1/3/0)
#<data>.W,([d ₃₂ ,B],I,d ₁₆)	16(1/0/0)	21(1/0/0)	24(1/3/0)
#<data>.L,([d ₃₂ ,B],I,d ₁₆)	17(1/0/0)	23(1/0/0)	26(1/3/0)
#<data>.W,([d ₃₂ ,B],I,d ₃₂)	16(1/0/0)	21(1/0/0)	24(1/3/0)
#<data>.L,([d ₃₂ ,B],I,d ₃₂)	17(1/0/0)	23(1/0/0)	29(1/4/0)

B = Base address; 0, An, PC, Xn, An + Xn. Form does not affect timing.

I = Index; 0, Xn

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

8.2.5 Jump Effective Address

The jump effective address table indicates the number of clock periods needed for the processor to calculate the specified effective address. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Address Mode	Best Case	Cache Case	Worst Case
(An)	0(0/0/0)	2(0/0/0)	2(0/0/0)
(d ₁₆ ,An)	1(0/0/0)	4(0/0/0)	4(0/0/0)
(xxx).W	0(0/0/0)	2(0/0/0)	2(0/0/0)
(xxx).L	0(0/0/0)	2(0/0/0)	2(0/0/0)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	3(0/0/0)	6(0/0/0)	6(0/0/0)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	3(0/0/0)	6(0/0/0)	6(0/0/0)
(B)	3(0/0/0)	6(0/0/0)	6(0/0/0)
(B,d ₁₆)	5(0/0/0)	8(0/0/0)	8(0/1/0)
(B,d ₃₂)	9(0/0/0)	12(0/0/0)	12(0/1/0)
([B],I)	8(1/0/0)	11(1/0/0)	11(1/1/0)
([B],I,d ₁₆)	10(1/0/0)	13(1/0/0)	14(1/1/0)
([B],I,d ₃₂)	10(1/0/0)	13(1/0/0)	14(1/1/0)
([d ₁₆ ,B],I)	10(1/0/0)	13(1/0/0)	14(1/1/0)
([d ₁₆ ,B],I,d ₁₆)	12(1/0/0)	15(1/0/0)	17(1/1/0)
([d ₁₆ ,B],I,d ₃₂)	12(1/0/0)	15(1/0/0)	17(1/1/0)
([d ₃₂ ,B],I)	14(1/0/0)	17(1/0/0)	19(1/2/0)
([d ₃₂ ,B],I,d ₁₆)	16(1/0/0)	19(1/0/0)	21(1/2/0)
([d ₃₂ ,B],I,d ₃₂)	16(1/0/0)	19(1/0/0)	23(1/2/0)

B = Base address; 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

I = Index; 0, Xn

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

8.2.6 MOVE Instruction

The MOVE instruction table indicates the number of clock periods needed for the processor to fetch, calculate, and perform the MOVE or MOVEA with the specified source and destination effective addresses, including both levels of indirection on memory indirect addressing modes. No additional tables are needed to calculate the total effective execution time for the MOVE or MOVEA instruction. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

BEST CASE

Source Address Mode	Destination							
	An	Dn	(An)	(An)+	-(An)	(d ₁₆ ,An)	(xxx).W	(xxx).L
Rn	0(0/0/0)	0(0/0/0)	3(0/0/1)	4(0/0/1)	3(0/0/1)	3(0/0/1)	3(0/0/1)	5(0/0/1)
#<data>.B,W	0(0/0/0)	0(0/0/0)	3(0/0/1)	4(0/0/1)	3(0/0/1)	3(0/0/1)	3(0/0/1)	5(0/0/1)
#<data>.L	0(0/0/0)	0(0/0/0)	3(0/0/1)	4(0/0/1)	3(0/0/1)	3(0/0/1)	3(0/0/1)	5(0/0/1)
(An)	3(1/0/0)	3(1/0/0)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	8(1/0/1)
(An)+	4(1/0/0)	4(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
-(An)	3(1/0/0)	3(1/0/0)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	8(1/0/1)
(d ₁₆ ,An) or (d ₁₆ ,PC)	3(1/0/0)	3(1/0/0)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	8(1/0/1)
(xxx).W	3(1/0/0)	3(1/0/0)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	8(1/0/1)
(xxx).L	3(1/0/0)	3(1/0/0)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	6(1/0/1)	8(1/0/1)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	4(1/0/0)	4(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	4(1/0/0)	4(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
(B)	4(1/0/0)	4(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
(d ₁₆ ,B)	6(1/0/0)	6(1/0/0)	9(1/0/1)	9(1/0/1)	9(1/0/1)	9(1/0/1)	9(1/0/1)	11(1/0/1)
(d ₃₂ ,B)	10(1/0/0)	10(1/0/0)	13(1/0/1)	13(1/0/1)	13(1/0/1)	13(1/0/1)	13(1/0/1)	15(1/0/1)
([B],l)	9(2/0/0)	9(2/0/0)	12(2/0/1)	12(2/0/1)	12(2/0/1)	12(2/0/1)	12(2/0/1)	14(2/0/1)
([B],l,d ₁₆)	11(2/0/0)	11(2/0/0)	14(2/0/1)	14(2/0/1)	14(2/0/1)	14(2/0/1)	14(2/0/1)	16(2/0/1)
([B],l,d ₃₂)	11(2/0/0)	11(2/0/0)	14(2/0/1)	14(2/0/1)	14(2/0/1)	14(2/0/1)	14(2/0/1)	16(2/0/1)
([d ₁₆ ,B],l)	11(2/0/0)	11(2/0/0)	14(2/0/1)	14(2/0/1)	14(2/0/1)	14(2/0/1)	14(2/0/1)	16(2/0/1)
([d ₁₆ ,B],l,d ₁₆)	13(2/0/0)	13(2/0/0)	16(2/0/1)	16(2/0/1)	16(2/0/1)	16(2/0/1)	16(2/0/1)	18(2/0/1)
([d ₁₆ ,B],d ₃₂)	13(2/0/0)	13(2/0/0)	16(2/0/1)	16(2/0/1)	16(2/0/1)	16(2/0/1)	16(2/0/1)	18(2/0/1)
([d ₃₂ ,B],l)	15(2/0/0)	15(2/0/0)	18(2/0/1)	18(2/0/1)	18(2/0/1)	18(2/0/1)	18(2/0/1)	20(2/0/1)
([d ₃₂ ,B],l,d ₁₆)	17(2/0/0)	17(2/0/0)	20(2/0/1)	20(2/0/1)	20(2/0/1)	20(2/0/1)	20(2/0/1)	22(2/0/1)
([d ₃₂ ,B],l,d ₃₂)	17(2/0/0)	17(2/0/0)	20(2/0/1)	20(2/0/1)	20(2/0/1)	20(2/0/1)	20(2/0/1)	22(2/0/1)

BEST CASE (Continued)

Source Address Mode	Destination							
	(d ₈ ,An,Xn)	(d ₁₆ ,An,Xn)	(B)	(d ₁₆ ,B)	(d ₃₂ ,B)	([B],l)	([B],l,d ₁₆)	([B],l,d ₃₂)
Rn	4(0/0/1)	6(0/0/1)	5(0/0/1)	7(0/0/1)	11(0/0/1)	9(1/0/1)	11(1/0/1)	12(1/0/1)
#<data>.B,W	4(0/0/1)	6(0/0/1)	5(0/0/1)	7(0/0/1)	11(0/0/1)	9(1/0/1)	11(1/0/1)	12(1/0/1)
#<data>.L	4(0/0/1)	6(0/0/1)	5(0/0/1)	7(0/0/1)	11(0/0/1)	9(1/0/1)	11(1/0/1)	12(1/0/1)
(An)	8(1/0/1)	10(1/0/1)	9(1/0/1)	11(1/0/1)	15(1/0/1)	13(2/0/1)	15(2/0/1)	16(2/0/1)
(An)+	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
-(An)	8(1/0/1)	10(1/0/1)	9(1/0/1)	11(1/0/1)	15(1/0/1)	13(2/0/1)	15(2/0/1)	16(2/0/1)
(d ₁₆ ,An) or (d ₁₆ ,PC)	8(1/0/1)	10(1/0/1)	9(1/0/1)	11(1/0/1)	15(1/0/1)	13(2/0/1)	15(2/0/1)	16(2/0/1)
(xxx).W	8(1/0/1)	10(1/0/1)	9(1/0/1)	11(1/0/1)	15(1/0/1)	13(2/0/1)	15(2/0/1)	16(2/0/1)
(xxx).L	8(1/0/1)	10(1/0/1)	9(1/0/1)	11(1/0/1)	15(1/0/1)	13(2/0/1)	15(2/0/1)	16(2/0/1)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	9(1/0/1)	10(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
(B)	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
(d ₁₆ ,B)	11(1/0/1)	13(1/0/1)	12(1/0/1)	14(1/0/1)	18(1/0/1)	16(2/0/1)	18(2/0/1)	19(2/0/1)
(d ₃₂ ,B)	15(1/0/1)	17(1/0/1)	18(1/0/1)	18(1/0/1)	22(1/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
([B],l)	14(2/0/1)	16(2/0/1)	17(2/0/1)	17(2/0/1)	21(2/0/1)	19(3/0/1)	21(3/0/1)	22(3/0/1)
([B],l,d ₁₆)	16(2/0/1)	18(2/0/1)	19(2/0/1)	19(2/0/1)	23(2/0/1)	21(3/0/1)	23(3/0/1)	24(3/0/1)
([B],l,d ₃₂)	16(2/0/1)	18(2/0/1)	19(2/0/1)	19(2/0/1)	23(2/0/1)	21(3/0/1)	23(3/0/1)	24(3/0/1)
([d ₁₆ ,B],l)	16(2/0/1)	18(2/0/1)	19(2/0/1)	19(2/0/1)	23(2/0/1)	21(3/0/1)	23(3/0/1)	24(3/0/1)
([d ₁₆ ,B],l,d ₁₆)	18(2/0/1)	20(2/0/1)	21(2/0/1)	21(2/0/1)	25(2/0/1)	23(3/0/1)	25(3/0/1)	26(3/0/1)
([d ₁₆ ,B],l,d ₃₂)	18(2/0/1)	20(2/0/1)	21(2/0/1)	21(2/0/1)	25(2/0/1)	23(3/0/1)	25(3/0/1)	26(3/0/1)
([d ₃₂ ,B],l)	20(2/0/1)	22(2/0/1)	23(2/0/1)	23(2/0/1)	27(2/0/1)	25(3/0/1)	27(3/0/1)	28(3/0/1)
([d ₃₂ ,B],l,d ₁₆)	22(2/0/1)	24(2/0/1)	25(2/0/1)	25(2/0/1)	29(2/0/1)	27(3/0/1)	29(3/0/1)	30(3/0/1)
([d ₃₂ ,B],l,d ₃₂)	22(2/0/1)	24(2/0/1)	25(2/0/1)	25(2/0/1)	29(2/0/1)	27(3/0/1)	29(3/0/1)	30(3/0/1)

BEST CASE (Concluded)

Source Address Mode	Destination					
	$[(d_{16}, B], l)$	$[(d_{16}, B], l, d_{16})$	$[(d_{16}, B], l, d_{32})$	$[(d_{32}, B], l)$	$[(d_{32}, B], l, d_{16})$	$[(d_{32}, B], l, d_{32})$
Rn	11(1/0/1)	13(1/0/1)	14(1/0/1)	15(1/0/1)	17(1/0/1)	18(1/0/1)
#<data>.B,W	11(1/0/1)	13(1/0/1)	14(1/0/1)	15(1/0/1)	17(1/0/1)	18(1/0/1)
#<data>.L	11(1/0/1)	13(1/0/1)	14(1/0/1)	15(1/0/1)	17(1/0/1)	18(1/0/1)
(An)	15(2/0/1)	17(2/0/1)	18(2/0/1)	19(2/0/1)	21(2/0/1)	22(2/0/1)
(An)+	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
-(An)	15(2/0/1)	17(2/0/1)	18(2/0/1)	19(2/0/1)	21(2/0/1)	22(2/0/1)
(d_{16}, An) or (d_{16}, PC)	15(2/0/1)	17(2/0/1)	18(2/0/1)	19(2/0/1)	21(2/0/1)	22(2/0/1)
(xxx).W	15(2/0/1)	17(2/0/1)	18(2/0/1)	19(2/0/1)	21(2/0/1)	22(2/0/1)
(xxx).L	15(2/0/1)	17(2/0/1)	18(2/0/1)	19(2/0/1)	21(2/0/1)	22(2/0/1)
(d_8, An, Xn) or (d_8, PC, Xn)	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
(d_{16}, An, Xn) or (d_{16}, PC, Xn)	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
(B)	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
(d_{16}, B)	18(2/0/1)	20(2/0/1)	21(2/0/1)	22(2/0/1)	24(2/0/1)	25(2/0/1)
(d_{32}, B)	22(2/0/1)	24(2/0/1)	25(2/0/1)	26(2/0/1)	28(2/0/1)	29(2/0/1)
$[(B], l)$	21(3/0/1)	23(3/0/1)	24(3/0/1)	25(3/0/1)	27(3/0/1)	28(3/0/1)
$[(B], l, d_{16})$	23(3/0/1)	25(3/0/1)	26(3/0/1)	27(3/0/1)	29(3/0/1)	30(3/0/1)
$[(B], l, d_{32})$	23(3/0/1)	25(3/0/1)	26(3/0/1)	27(3/0/1)	29(3/0/1)	30(3/0/1)
$[(d_{16}, B], l)$	23(3/0/1)	25(3/0/1)	26(3/0/1)	27(3/0/1)	29(3/0/1)	30(3/0/1)
$[(d_{16}, B], l, d_{16})$	25(3/0/1)	27(3/0/1)	28(3/0/1)	29(3/0/1)	31(3/0/1)	32(3/0/1)
$[(d_{16}, B], l, d_{32})$	25(3/0/1)	27(3/0/1)	28(3/0/1)	29(3/0/1)	31(3/0/1)	32(0/0/1)
$[(d_{32}, B], l)$	27(3/0/1)	29(3/0/1)	30(3/0/1)	31(3/0/1)	33(3/0/1)	34(3/0/1)
$[(d_{32}, B], l, d_{16})$	29(3/0/1)	31(3/0/1)	32(3/0/1)	33(3/0/1)	35(3/0/1)	36(3/0/1)
$[(d_{32}, B], l, d_{32})$	29(3/0/1)	31(3/0/1)	32(3/0/1)	33(3/0/1)	35(3/0/1)	36(3/0/1)

CACHE CASE

Source Address Mode	Destination							
	An	Dn	(An)	(An)+	-(An)	(d ₁₆ , An)	(xxx).W	(xxx).L
Rn	2(0/0/0)	2(0/0/0)	4(0/0/1)	4(0/0/1)	5(0/0/1)	5(0/0/1)	4(0/0/1)	6(0/0/1)
#<data>.B,W	4(0/0/0)	4(0/0/0)	6(0/0/1)	6(0/0/1)	7(0/0/1)	7(0/0/1)	6(0/0/1)	8(0/0/1)
#<data>.L	6(0/0/0)	6(0/0/0)	8(0/0/1)	8(0/0/1)	9(0/0/1)	9(0/0/1)	8(0/0/1)	10(0/0/1)
(An)	6(1/0/0)	6(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
(An)+	6(1/0/0)	6(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
-(An)	7(1/0/0)	7(1/0/0)	8(1/0/1)	8(1/0/1)	8(1/0/1)	8(1/0/1)	8(1/0/1)	10(1/0/1)
(d ₁₆ ,An) or (d ₁₆ ,PC)	7(1/0/0)	7(1/0/0)	8(1/0/1)	8(1/0/1)	8(1/0/1)	8(1/0/1)	8(1/0/1)	10(1/0/1)
(xxx).W	6(1/0/0)	6(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
(xxx).L	6(1/0/0)	6(1/0/0)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	7(1/0/1)	9(1/0/1)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	9(1/0/0)	9(1/0/0)	10(1/0/1)	10(1/0/1)	10(1/0/1)	10(1/0/1)	10(1/0/1)	12(1/0/1)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	9(1/0/0)	9(1/0/0)	10(1/0/1)	10(1/0/1)	10(1/0/1)	10(1/0/1)	10(1/0/1)	12(1/0/1)
(B)	9(1/0/0)	9(1/0/0)	10(1/0/1)	10(1/0/1)	10(1/0/1)	10(1/0/1)	10(1/0/1)	12(1/0/1)
(d ₁₆ ,B)	11(1/0/0)	11(1/0/0)	12(1/0/1)	12(1/0/1)	12(1/0/1)	12(1/0/1)	12(1/0/1)	14(1/0/1)
(d ₃₂ ,B)	15(1/0/0)	15(1/0/0)	16(1/0/1)	16(1/0/1)	16(1/0/1)	16(1/0/1)	16(1/0/1)	18(1/0/1)
([B],l)	14(2/0/0)	14(2/0/0)	15(2/0/1)	15(2/0/1)	15(2/0/1)	15(2/0/1)	15(2/0/1)	17(2/0/1)
([B],l,d ₁₆)	16(2/0/0)	16(2/0/0)	17(2/0/1)	17(2/0/1)	17(2/0/1)	17(2/0/1)	17(2/0/1)	19(2/0/1)
([B],l,d ₃₂)	16(2/0/0)	16(2/0/0)	17(2/0/1)	17(2/0/1)	17(2/0/1)	17(2/0/1)	17(2/0/1)	19(2/0/1)
([d ₁₆ ,B],l)	16(2/0/0)	16(2/0/0)	17(2/0/1)	17(2/0/1)	17(2/0/1)	17(2/0/1)	17(2/0/1)	19(2/0/1)
([d ₁₆ ,B],l,d ₁₆)	18(2/0/0)	18(2/0/0)	19(2/0/1)	19(2/0/1)	19(2/0/1)	19(2/0/1)	19(2/0/1)	21(2/0/1)
([d ₁₆ ,B],l,d ₃₂)	18(2/0/0)	18(2/0/0)	19(2/0/1)	19(2/0/1)	19(2/0/1)	19(2/0/1)	19(2/0/1)	21(2/0/1)
([d ₃₂ ,B],l)	20(2/0/0)	20(2/0/0)	21(2/0/1)	21(2/0/1)	21(2/0/1)	21(2/0/1)	21(2/0/1)	23(2/0/1)
([d ₃₂ ,B],l,d ₁₆)	22(2/0/0)	22(2/0/0)	23(2/0/1)	23(2/0/1)	23(2/0/1)	23(2/0/1)	23(2/0/1)	25(2/0/1)
([d ₃₂ ,B],l,d ₃₂)	22(2/0/0)	22(2/0/0)	23(2/0/1)	23(2/0/1)	23(2/0/1)	23(2/0/1)	23(2/0/1)	25(2/0/1)

CACHE CASE (Continued)

Source Address Mode	Destination							
	(d ₈ ,An,Xn)	(d ₁₆ ,An,Xn)	(B)	(d ₁₆ ,B)	(d ₃₂ ,B)	([B],l)	([B],l,d ₁₆)	([B],l,d ₃₂)
Rn	7(0/0/1)	9(0/0/1)	8(0/0/1)	10(0/0/1)	14(0/0/1)	12(1/0/1)	14(1/0/1)	15(1/0/1)
#<data>.B,W	7(0/0/1)	9(0/0/1)	8(0/0/1)	10(0/0/1)	14(0/0/1)	12(1/0/1)	14(1/0/1)	15(1/0/1)
#<data>.L	9(0/0/1)	11(0/0/1)	10(0/0/1)	12(0/0/1)	16(0/0/1)	14(1/0/1)	16(1/0/1)	17(1/0/1)
(An)	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
(An)+	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
-(An)	10(1/0/1)	12(1/0/1)	11(1/0/1)	13(1/0/1)	17(1/0/1)	15(2/0/1)	17(2/0/1)	18(2/0/1)
(d ₁₆ ,An) or (d ₁₆ ,PC)	10(1/0/1)	12(2/0/1)	11(1/0/1)	13(1/0/1)	17(1/0/1)	15(2/0/1)	17(2/0/1)	18(2/0/1)
(xxx).W	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
(xxx).L	9(1/0/1)	11(1/0/1)	10(1/0/1)	12(1/0/1)	16(1/0/1)	14(2/0/1)	16(2/0/1)	17(2/0/1)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	12(1/0/1)	14(1/0/1)	13(1/0/1)	15(1/0/1)	19(1/0/1)	17(2/0/1)	19(2/0/1)	20(2/0/1)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	12(1/0/1)	14(1/0/1)	13(1/0/1)	15(1/0/1)	19(1/0/1)	17(2/0/1)	19(2/0/1)	20(2/0/1)
(B)	12(1/0/1)	14(1/0/1)	13(1/0/1)	15(1/0/1)	19(1/0/1)	17(2/0/1)	19(2/0/1)	20(2/0/1)
(d ₁₆ ,B)	14(1/0/1)	16(1/0/1)	15(1/0/1)	17(1/0/1)	21(1/0/1)	19(2/0/1)	21(2/0/1)	22(2/0/1)
(d ₃₂ ,B)	18(1/0/1)	20(1/0/1)	19(1/0/1)	21(1/0/1)	25(1/0/1)	23(2/0/1)	25(2/0/1)	26(2/0/1)
([B],l)	17(2/0/1)	19(2/0/1)	18(2/0/1)	20(2/0/1)	24(2/0/1)	22(3/0/1)	24(3/0/1)	25(3/0/1)
([B],l,d ₁₆)	19(2/0/1)	21(2/0/1)	20(2/0/1)	22(2/0/1)	26(2/0/1)	24(3/0/1)	26(3/0/1)	27(3/0/1)
([B],l,d ₃₂)	19(2/0/1)	21(2/0/1)	20(2/0/1)	22(2/0/1)	26(2/0/1)	24(3/0/1)	26(3/0/1)	27(3/0/1)
([d ₁₆ ,B],l)	19(2/0/1)	21(2/0/1)	20(2/0/1)	22(2/0/1)	26(2/0/1)	24(3/0/1)	26(3/0/1)	27(3/0/1)
([d ₁₆ ,B],l,d ₁₆)	21(2/0/1)	23(2/0/1)	22(2/0/1)	24(2/0/1)	28(2/0/1)	26(3/0/1)	28(3/0/1)	29(3/0/1)
([d ₁₆ ,B],l,d ₃₂)	21(2/0/1)	23(2/0/1)	22(2/0/1)	24(2/0/1)	28(2/0/1)	26(3/0/1)	28(3/0/1)	29(3/0/1)
([d ₃₂ ,B],l)	23(2/0/1)	25(2/0/1)	24(2/0/1)	26(2/0/1)	30(2/0/1)	28(3/0/1)	30(3/0/1)	31(3/0/1)
([d ₃₂ ,B],l,d ₁₆)	25(2/0/1)	27(2/0/1)	26(2/0/1)	28(2/0/1)	32(2/0/1)	30(3/0/1)	32(3/0/1)	33(3/0/1)
([d ₃₂ ,B],l,d ₃₂)	25(2/0/1)	27(2/0/1)	26(2/0/1)	28(2/0/1)	32(2/0/1)	30(3/0/1)	32(3/0/1)	33(3/0/1)

CACHE CASE (Concluded)

Source Address Mode	Destination					
	$[[d_{16}, B], l]$	$[[d_{16}, B], l, d_{16}]$	$[[d_{16}, B], l, d_{32}]$	$[[d_{32}, B], l]$	$[[d_{32}, B], l, d_{16}]$	$[[d_{32}, B], l, d_{32}]$
Rn	14(1/0/1)	16(1/0/1)	17(1/0/1)	18(1/0/1)	20(1/0/1)	21(1/0/1)
#<data>.B,W	14(1/0/1)	16(1/0/1)	17(1/0/1)	18(1/0/1)	20(1/0/1)	21(1/0/1)
#<data>.L	16(1/0/1)	18(1/0/1)	19(1/0/1)	20(1/0/1)	22(1/0/1)	23(1/0/1)
(An)	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
(An)+	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
-(An)	17(2/0/1)	19(2/0/1)	20(2/0/1)	21(2/0/1)	23(2/0/1)	24(2/0/1)
(d_{16}, An) or (d_{16}, PC)	17(2/0/1)	19(2/0/1)	20(2/0/1)	21(2/0/1)	23(2/0/1)	24(2/0/1)
(xxx).W	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
(xxx).L	16(2/0/1)	18(2/0/1)	19(2/0/1)	20(2/0/1)	22(2/0/1)	23(2/0/1)
(d_8, An, Xn) or (d_8, PC, Xn)	19(2/0/1)	21(2/0/1)	22(2/0/1)	23(2/0/1)	25(2/0/1)	26(2/0/1)
(d_{16}, An, Xn) or (d_{16}, PC, Xn)	19(2/0/1)	21(2/0/1)	22(2/0/1)	23(2/0/1)	25(2/0/1)	26(2/0/1)
(B)	19(2/0/1)	21(2/0/1)	22(2/0/1)	23(2/0/1)	25(2/0/1)	26(2/0/1)
(d_{16}, B)	21(2/0/1)	23(2/0/1)	24(2/0/1)	25(2/0/1)	27(2/0/1)	28(2/0/1)
(d_{32}, B)	25(2/0/1)	27(2/0/1)	28(2/0/1)	29(2/0/1)	31(2/0/1)	32(2/0/1)
$[[B], l]$	24(3/0/1)	26(3/0/1)	27(3/0/1)	28(3/0/1)	30(3/0/1)	31(3/0/1)
$[[B], l, d_{16}]$	26(3/0/1)	28(3/0/1)	29(3/0/1)	30(3/0/1)	32(3/0/1)	33(3/0/1)
$[[B], l, d_{32}]$	26(3/0/1)	28(3/0/1)	29(3/0/1)	30(3/0/1)	32(3/0/1)	33(3/0/1)
$[[d_{16}, B], l]$	26(3/0/1)	28(3/0/1)	29(3/0/1)	30(3/0/1)	32(3/0/1)	33(3/0/1)
$[[d_{16}, B], l, d_{16}]$	28(3/0/1)	30(3/0/1)	31(3/0/1)	32(3/0/1)	34(3/0/1)	35(3/0/1)
$[[d_{16}, B], l, d_{32}]$	28(3/0/1)	30(3/0/1)	31(3/0/1)	32(3/0/1)	34(3/0/1)	35(3/0/1)
$[[d_{32}, B], l]$	30(3/0/1)	32(3/0/1)	33(3/0/1)	34(3/0/1)	36(3/0/1)	37(3/0/1)
$[[d_{32}, B], l, d_{16}]$	32(3/0/1)	34(3/0/1)	35(3/0/1)	36(3/0/1)	38(3/0/1)	39(3/0/1)
$[[d_{32}, B], l, d_{32}]$	32(3/0/1)	34(3/0/1)	35(3/0/1)	36(3/0/1)	38(3/0/1)	39(3/0/1)

WORST CASE

Source Address Mode	Destination							
	An	Dn	(An)	(An)+	-(An)	(d ₁₆ ,An)	(xxx).W	(xxx).L
Rn	3(0/1/0)	3(0/1/0)	5(0/1/0)	5(0/1/1)	6(0/1/1)	7(0/1/1)	7(0/1/1)	9(0/2/1)
#<data>.B,W	3(0/1/0)	3(0/1/0)	5(0/1/0)	8(0/1/1)	6(0/1/1)	7(0/1/1)	7(0/1/1)	9(0/2/1)
#<data>.L	5(0/1/0)	5(0/1/0)	7(0/0/1)	7(0/1/1)	8(0/1/1)	9(0/1/1)	9(0/1/1)	11(0/2/1)
(An)	7(1/1/0)	7(1/1/0)	9(1/1/1)	9(1/1/1)	9(1/1/1)	11(1/1/1)	11(1/1/1)	13(1/2/1)
(An)+	7(1/1/0)	7(1/1/0)	9(1/1/1)	9(1/1/1)	9(1/1/1)	11(1/1/1)	11(1/1/1)	13(1/2/1)
-(An)	8(1/1/0)	8(1/1/0)	10(1/1/1)	10(1/1/1)	10(1/1/1)	12(1/1/1)	12(1/1/1)	14(1/2/1)
(d ₁₆ ,An) or (d ₁₆ ,PC)	9(1/2/0)	9(1/2/0)	11(1/2/1)	11(1/2/1)	11(1/2/1)	13(1/2/1)	13(1/2/1)	15(1/3/1)
(xxx).W	8(1/2/0)	8(1/2/0)	10(1/2/1)	10(1/2/1)	10(1/2/1)	12(1/2/1)	12(1/2/1)	14(1/3/1)
(xxx).L	10(1/2/0)	10(1/2/0)	12(1/2/1)	12(1/2/1)	12(1/2/1)	14(1/2/1)	14(1/2/1)	16(1/3/1)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	11(1/2/0)	11(1/2/0)	13(1/2/1)	13(1/2/1)	13(1/2/1)	15(1/2/1)	15(1/2/1)	17(1/3/1)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	12(1/2/0)	12(1/2/0)	14(1/2/1)	14(1/2/1)	14(1/2/1)	16(1/2/1)	16(1/2/1)	18(1/3/1)
(B)	12(1/2/0)	12(1/2/0)	14(1/2/1)	14(1/2/1)	14(1/2/1)	16(1/2/1)	16(1/2/1)	18(1/3/1)
(d ₁₆ ,B)	15(1/2/0)	15(1/2/0)	17(1/2/1)	17(1/2/1)	17(1/3/1)	19(1/2/1)	19(1/2/1)	21(1/3/1)
(d ₃₂ ,B)	19(1/3/0)	19(1/3/0)	21(1/3/1)	21(1/3/1)	21(1/3/1)	23(1/3/1)	23(1/3/1)	25(1/4/1)
([B],l)	16(2/2/0)	16(2/2/0)	18(2/2/1)	18(2/2/1)	18(2/2/1)	20(2/2/1)	20(2/2/1)	22(2/3/1)
([B],l,d ₁₆)	19(2/2/0)	19(2/2/0)	21(2/2/1)	21(2/2/1)	21(2/2/1)	23(2/2/1)	23(2/2/1)	25(2/3/1)
([B],l,d ₃₂)	20(2/3/0)	20(2/3/0)	22(2/3/1)	22(2/3/1)	22(2/3/1)	24(2/3/1)	24(2/3/1)	26(2/4/1)
([d ₁₆ ,B],l)	19(2/2/0)	19(2/2/0)	21(2/2/1)	21(2/2/1)	21(2/2/1)	23(2/2/1)	23(2/2/1)	25(2/3/1)
([d ₁₆ ,B],l,d ₁₆)	22(2/3/0)	22(2/3/0)	24(2/3/1)	24(2/3/1)	24(2/3/1)	26(2/3/1)	26(2/3/1)	28(2/4/1)
([d ₁₆ ,B],d ₃₂)	23(2/3/0)	23(2/3/0)	25(2/3/1)	25(2/3/1)	25(2/3/1)	27(2/3/1)	27(2/3/1)	29(2/4/1)
([d ₃₂ ,B],l)	23(2/3/0)	23(2/3/0)	25(2/3/1)	25(2/3/1)	25(2/3/1)	27(2/3/1)	27(2/3/1)	29(2/4/1)
([d ₃₂ ,B],l,d ₁₆)	25(2/3/0)	25(2/3/0)	27(2/3/1)	27(2/3/1)	27(2/3/1)	29(2/3/1)	29(2/3/1)	31(2/4/1)
([d ₃₂ ,B],l,d ₃₂)	27(2/4/0)	27(2/4/0)	29(2/4/1)	29(2/4/1)	29(2/4/1)	31(2/4/1)	31(2/4/1)	33(2/5/1)

WORST CASE (Continued)

Source Address Mode	Destination							
	(d ₈ ,An,Xn)	(d ₁₆ ,An,Xn)	(B)	(d ₁₆ ,B)	(d ₃₂ ,B)	([B],l)	([B],l,d ₁₆)	([B],l,D ₃₂)
Rn	9(0/1/1)	12(0/2/1)	10(0/1/1)	14(0/2/1)	19(0/2/1)	14(1/1/1)	17(1/2/1)	20(1/2/1)
#<data>.B,W	9(0/1/1)	12(0/2/1)	10(0/1/1)	14(0/2/1)	19(0/2/1)	14(1/1/1)	17(1/2/1)	20(1/2/1)
#<data>.L	11(0/1/1)	14(0/2/1)	12(0/1/1)	16(0/2/1)	21(0/2/1)	16(1/1/1)	19(1/2/1)	22(1/2/1)
(An)	11(1/1/1)	14(1/2/1)	12(1/1/1)	16(1/2/1)	21(1/2/1)	12(2/1/1)	19(2/2/1)	22(2/2/1)
(An)+	11(1/1/1)	14(1/2/1)	12(1/1/1)	16(1/2/1)	21(1/2/1)	12(2/1/1)	19(2/2/1)	22(2/2/1)
-(An)	12(1/1/1)	15(1/2/1)	13(1/1/1)	17(1/2/1)	22(1/2/1)	13(2/1/1)	20(2/2/1)	23(2/2/1)
(d ₁₆ ,An) or (d ₁₆ ,PC)	13(1/2/1)	16(1/3/1)	14(1/2/1)	18(1/3/1)	23(1/3/1)	14(2/2/1)	21(2/3/1)	24(2/3/1)
(xxx).W	12(1/2/1)	15(1/3/1)	13(1/2/1)	17(1/3/1)	22(1/3/1)	13(2/2/1)	20(2/3/1)	23(2/3/1)
(xxx).L	14(1/2/1)	17(1/3/1)	15(1/2/1)	19(1/3/1)	24(1/3/1)	15(2/2/1)	22(2/3/1)	25(2/3/1)
(d ₈ ,An,Xn) or (d ₈ ,PC,Xn)	15(1/2/1)	18(1/3/1)	16(1/2/1)	20(1/3/1)	25(1/3/1)	16(2/2/1)	23(2/3/1)	26(2/3/1)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	16(1/2/1)	19(1/3/1)	17(1/2/1)	21(1/3/1)	26(1/3/1)	17(2/2/1)	24(2/3/1)	27(2/3/1)
(B)	16(1/2/1)	19(1/3/1)	17(1/2/1)	21(1/3/1)	26(1/3/1)	17(2/2/1)	24(2/3/1)	27(2/3/1)
(d ₁₆ ,B)	19(1/2/1)	22(1/3/1)	20(1/2/1)	24(1/3/1)	29(1/3/1)	20(2/2/1)	27(2/3/1)	30(2/3/1)
(d ₃₂ ,B)	23(1/3/1)	26(1/4/1)	24(1/3/1)	28(1/4/1)	33(1/4/1)	24(2/3/1)	31(2/4/1)	34(2/4/1)
([B],l)	20(2/2/1)	23(2/3/1)	21(2/2/1)	25(2/3/1)	30(2/3/1)	21(3/2/1)	28(3/3/1)	31(3/3/1)
([B],l,d ₁₆)	23(2/2/1)	26(2/3/1)	24(2/2/1)	28(2/3/1)	33(2/3/1)	24(3/2/1)	31(3/3/1)	34(3/3/1)
([B],l,d ₃₂)	24(2/3/1)	27(2/4/1)	25(2/3/1)	29(2/4/1)	34(2/4/1)	25(3/3/1)	32(3/4/1)	35(3/4/1)
([d ₁₆ ,B],l)	23(2/2/1)	26(2/3/1)	24(2/2/1)	28(2/3/1)	33(2/3/1)	24(3/2/1)	31(3/3/1)	34(3/3/1)
([d ₁₆ ,B],l,d ₁₆)	26(2/3/1)	29(2/4/1)	27(2/3/1)	31(2/4/1)	36(2/4/1)	27(3/3/1)	34(3/4/1)	37(3/4/1)
([d ₁₆ ,B],l,d ₃₂)	27(2/3/1)	30(2/4/1)	28(2/3/1)	32(2/4/1)	37(2/4/1)	28(3/3/1)	35(3/4/1)	38(3/4/1)
([d ₃₂ ,B],l)	27(2/3/1)	30(2/4/1)	28(2/3/1)	32(2/4/1)	37(2/4/1)	28(3/3/1)	35(3/4/1)	38(3/4/1)
([d ₃₂ ,B],l,d ₁₆)	29(2/3/1)	32(2/4/1)	30(2/3/1)	34(2/4/1)	39(2/4/1)	30(3/3/1)	37(3/4/1)	40(3/4/1)
([d ₃₂ ,B],l,d ₃₂)	31(2/4/1)	34(2/5/1)	32(2/4/1)	36(2/5/1)	41(2/5/1)	32(3/4/1)	39(3/5/1)	42(3/5/1)

WORST CASE (Concluded)

Source Address Mode	Destination					
	$[(d_{16},B),l]$	$[(d_{16},B),l,d_{16}]$	$[(d_{16},B),l,d_{32}]$	$[(d_{32},B),l]$	$[(d_{32},B),l,d_{16}]$	$[(d_{32},B),l,d_{32}]$
Rn	17(1/2/1)	20(1/2/1)	23(1/3/1)	22(1/2/1)	25(1/3/1)	27(1/3/1)
#<data>.B,W	17(1/2/1)	20(1/2/1)	23(1/3/1)	22(1/2/1)	25(1/3/1)	27(1/3/1)
#<data>.L	19(1/2/1)	22(1/2/1)	25(1/3/1)	24(1/2/1)	27(1/3/1)	29(1/3/1)
(An)	19(2/2/1)	22(2/2/1)	25(2/3/1)	24(2/2/1)	27(2/3/1)	29(2/3/1)
(An)+	19(2/2/1)	22(2/2/1)	25(2/3/1)	24(2/2/1)	27(2/3/1)	29(2/3/1)
-(An)	20(2/2/1)	23(2/2/1)	26(2/3/1)	25(2/2/1)	28(2/3/1)	30(2/3/1)
(d_{16},An) or (d_{16},PC)	21(2/3/1)	24(2/3/1)	27(2/4/1)	26(2/3/1)	29(2/4/1)	31(2/4/1)
(xxx).W	20(2/3/1)	23(2/3/1)	26(2/4/1)	27(2/3/1)	28(2/4/1)	30(2/4/1)
(xxx).L	22(2/3/1)	25(2/3/1)	28(2/4/1)	29(2/3/1)	30(2/4/1)	32(2/4/1)
(d_8,An,Xn) or (d_8,PC,Xn)	23(2/3/1)	26(2/3/1)	29(2/4/1)	30(2/3/1)	31(2/4/1)	33(2/4/1)
(d_{16},An,Xn) or (d_{16},PC,Xn)	24(2/3/1)	27(2/3/1)	30(2/4/1)	31(2/3/1)	32(2/4/1)	34(2/4/1)
(B)	24(2/3/1)	27(2/3/1)	30(2/4/1)	31(2/3/1)	32(2/4/1)	34(2/4/1)
(d_{16},B)	27(2/3/1)	30(2/3/1)	33(2/4/1)	34(2/3/1)	35(2/4/1)	37(2/4/1)
(d_{32},B)	31(2/4/1)	34(2/4/1)	37(2/5/1)	38(2/4/1)	39(2/5/1)	41(2/5/1)
$[(B),l]$	28(3/3/1)	31(3/3/1)	34(3/4/1)	35(3/3/1)	36(3/4/1)	38(3/4/1)
$[(B),l,d_{16}]$	31(3/3/1)	34(3/3/1)	37(3/4/1)	38(3/3/1)	39(3/4/1)	41(3/4/1)
$[(B),l,d_{32}]$	32(3/4/1)	35(3/4/1)	38(3/5/1)	39(3/4/1)	40(3/5/1)	42(3/5/1)
$[(d_{16},B),l]$	31(3/3/1)	34(3/3/1)	37(3/4/1)	38(3/3/1)	39(3/4/1)	41(3/4/1)
$[(d_{16},B),l,d_{16}]$	34(3/4/1)	37(3/4/1)	40(3/5/1)	41(3/4/1)	42(3/5/1)	44(3/5/1)
$[(d_{16},B),l,d_{32}]$	35(3/4/1)	38(3/4/1)	41(3/5/1)	42(3/4/1)	43(3/5/1)	45(3/5/1)
$[(d_{32},B),l]$	35(3/4/1)	38(3/4/1)	41(3/5/1)	42(3/4/1)	43(3/5/1)	45(3/5/1)
$[(d_{32},B),l,d_{16}]$	37(3/4/1)	40(3/4/1)	43(3/5/1)	44(3/4/1)	45(3/5/1)	47(3/5/1)
$[(d_{32},B),l,d_{32}]$	39(3/5/1)	42(3/5/1)	45(3/6/1)	46(3/5/1)	47(3/6/1)	49(3/6/1)

8.2.7 Special-Purpose MOVE Instruction

The special-purpose MOVE instruction table indicates the number of clock periods needed for the processor to fetch, calculate, and perform the special-purpose MOVE operation on the control registers or specified effective address. The total number of clock cycles is outside the parentheses, the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction		Best Case	Cache Case	Worst Case
EXG	Ry,Rx	0(0/0/0)	2(0/0/0)	3(0/1/0)
MOVEC	Cr,Rn	3(0/0/0)	6(0/0/0)	7(0/1/0)
MOVEC	Rn,Cr	9(0/0/0)	12(0/0/0)	13(0/1/0)
MOVE	PSW,Rn	1(0/0/0)	4(0/0/0)	5(0/1/0)
† MOVE	PSW,Mem	5(0/0/1)	5(0/0/1)	7(0/1/1)
* MOVE	EA,CCR	4(0/0/0)	4(0/0/0)	5(0/1/0)
* MOVE	EA,SR	8(0/0/0)	8(0/0/0)	11(0/2/0)
‡ MOVEM	EA,RL	8 + 4n (n/0/0)	8 + 4n (n/0/0)	9 + 4n (n/1/0)
‡ MOVEM	RL,EA	4 + 3n (0/0/n)	4 + 3n (0/0/n)	5 + 3n (0/1/n)
MOVEP.W	Dn,(d ₁₆ ,An)	8(0/0/2)	11(0/0/2)	11(0/1/2)
MOVEP.L	Dn,(d ₁₆ ,An)	14(0/0/4)	17(0/0/4)	17(0/1/4)
MOVEP.W	(d ₁₆ ,An),Dn	10(2/0/0)	12(2/0/0)	12(2/1/0)
MOVEP.L	(d ₁₆ ,An),Dn	16(4/0/0)	18(4/0/0)	18(4/1/0)
‡ MOVES	EA,Rn	7(1/0/0)	7(1/0/0)	8(1/1/0)
‡ MOVES	Rn,EA	5(0/0/1)	5(0/0/1)	7(0/1/1)
MOVE	USP	0(0/0/0)	2(0/0/0)	3(0/1/0)
SWAP	Rx,Ry	1(0/0/0)	4(0/0/0)	4(0/1/0)

n—Number of Registers to Transfer

RL—Register List

*Add Fetch Effective Address Time

†Add Calculate Effective Address Time

‡Add Calculate Immediate Address Time

8.2.8 Arithmetic/Logical Instructions

The arithmetic/logical instructions table indicates the number of clock periods needed for the processor to perform the specified arithmetic/logical operation using the specified addressing mode. It also includes, in worst case, the amount of time needed to prefetch the next instruction. Footnotes specify when to add either fetch address or fetch immediate effective address time. This sum gives the total effective execution time for the operation using the specified addressing mode. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction			Best Case	Cache Case	Worst Case
*	ADD	EA,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	ADDA	EA,An	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	ADD	Dn,EA	3(0/0/1)	4(0/0/1)	6(0/1/1)
*	AND	EA,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	AND	Dn,EA	3(0/0/1)	4(0/0/1)	6(0/1/1)
*	EOR	Dn,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	EOR	Dn,Mem	3(0/0/1)	4(0/0/1)	6(0/1/1)
*	OR	EA,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	OR	Dn,EA	3(0/0/1)	4(0/0/1)	6(0/1/1)
*	SUB	EA,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	SUBA	EA,An	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	SUB	Dn,EA	3(0/0/1)	4(0/0/1)	6(0/1/1)
*	CMP	EA,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	CMPA	EA,An	1(0/0/0)	4(0/0/0)	4(0/1/0)
**	CMP2	EA,Rn	16(1/0/0)	18(1/0/0)	18(1/1/0)
*	MUL.W	EA,Dn	25(0/0/0)	27(0/0/0)	28(0/1/0)
**	MUL.L	EA,Dn	41(0/0/0)	43(0/0/0)	44(0/1/0)
*	DIVU.W	EA,Dn	42(0/0/0)	44(0/0/0)	44(0/1/0)
**	DIVU.L	EA,Dn	76(0/0/0)	78(0/0/0)	79(0/1/0)
*	DIVS.W	EA,Dn	54(0/0/0)	56(0/0/0)	57(0/1/0)
**	DIVS.L	EA,Dn	88(0/0/0)	90(0/0/0)	91(0/1/0)

*Add Fetch Effective Address Time

**Add Fetch Immediate Address Time

8.2.9 Immediate Arithmetic/Logical Instructions

The immediate arithmetic/logical instructions table indicates the number of clock periods needed for the processor to fetch the source immediate data value and perform the specified arithmetic/logical operation using the specified destination addressing mode. Footnotes indicate when to add appropriate fetch effective or fetch immediate effective address time. This computation will give the total execution time needed to perform the appropriate immediate arithmetic/logical operation. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction		Best Case	Cache Case	Worst Case
MOVEQ	#<data>,Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
ADDQ	#<data>,Rn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	ADDQ	#<data>,Mem	4(0/0/1)	6(0/1/1)
	SUBQ	#<data>,Rn	2(0/0/0)	3(0/1/0)
*	SUBQ	#<data>,Mem	4(0/0/1)	6(0/1/1)
**	ADDI	#<data>,Dn	2(0/0/0)	3(0/1/0)
**	ADDI	#<data>,Mem	4(0/0/1)	6(0/1/1)
**	ANDI	#<data>,Dn	2(0/0/0)	3(0/1/0)
**	ANDI	#<data>,Mem	4(0/0/1)	6(0/1/1)
**	EORI	#<data>,Dn	2(0/0/0)	3(0/1/0)
**	EORI	#<data>,Mem	4(0/0/1)	6(0/1/1)
**	ORI	#<data>,Dn	2(0/0/0)	3(0/1/0)
**	ORI	#<data>,Mem	4(0/0/1)	6(0/1/1)
**	SUBI	#<data>,Dn	2(0/0/0)	3(0/1/0)
**	SUBI	#<data>,Mem	4(0/0/1)	6(0/1/1)
**	CMPI	#<data>,EA	2(0/0/0)	3(0/1/0)

*Add Fetch Effective Address Time

** Add Fetch Immediate Address Time

8.2.10 Binary-Coded Decimal Operations

The binary-coded decimal operations table indicates the number of clock periods needed for the processor to perform the specified operation using the given addressing modes, with complete execution times given. No additional tables are needed to calculate total effective execution time for these instructions. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction		Best Case	Cache Case	Worst Case
ABCD	Dn,Dn	4(0/0/0)	4(0/0/0)	5(0/1/0)
ABCD	-(An),-(An)	14(2/0/1)	16(2/0/1)	17(2/1/1)
SBCD	Dn,Dn	4(0/0/0)	4(0/0/0)	5(0/1/0)
SBCD	-(An),-(An)	14(2/0/1)	16(2/0/1)	17(2/1/1)
ADDX	Dn,Dn	2(0/0/0)	2(0/0/0)	3(0/1/0)
ADDX	-(An),-(An)	10(2/0/1)	12(2/0/1)	13(2/1/1)
SUBX	Dn,Dn	2(0/0/0)	2(0/0/0)	3(0/1/0)
SUBX	-(An),-(An)	10(2/0/1)	12(2/0/1)	13(2/1/1)
CMPM	(An)+,(An)+	8(2/0/0)	9(2/0/0)	10(2/1/0)
PACK	Dn,Dn,#<data>	3(0/0/0)	6(0/0/0)	7(0/1/0)
PACK	-(An),-(An),#<data>	11(1/0/1)	13(1/0/1)	13(1/1/1)
UNPK	Dn,Dn,#<data>	5(0/0/0)	8(0/0/0)	9(0/1/0)
UNPK	-(An),-(An),#<data>	11(1/0/1)	13(1/0/1)	13(1/1/1)

8.2.11 Single-Operand Instructions

The single-operand instructions table indicates the number of clock periods needed for the processor to perform the specified operation on the given addressing mode. Footnotes indicate when it is necessary to add another table entry to calculate the total effective execution time for the instruction. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction			Best Case	Cache Case	Worst Case
	CLR	Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
†	CLR	Mem	3(0/0/1)	4(0/0/1)	6(0/1/1)
	NEG	Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	NEG	Mem	3(0/0/1)	4(0/0/1)	6(0/1/1)
	NEGX	Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	NEGX	Mem	3(0/0/1)	4(0/0/1)	6(0/1/1)
	NOT	Dn	0(0/0/0)	2(0/0/0)	3(0/1/0)
*	NOT	Mem	3(0/0/1)	4(0/0/1)	6(0/1/1)
	EXT	Dn	1(0/0/0)	4(0/0/0)	4(0/1/0)
	NBCD	Dn	6(0/0/0)	6(0/0/0)	6(0/1/0)
	Scc	Dn	1(0/0/0)	4(0/0/0)	4(0/1/0)
†	Scc	Mem	6(0/0/1)	6(0/0/1)	6(0/1/1)
	TAS	Dn	1(0/0/0)	4(0/0/0)	4(0/1/0)
†	TAS	Mem	12(1/0/1)	12(1/0/1)	13(1/1/1)
*	TST	EA	0(0/0/0)	2(0/0/0)	3(0/1/0)

*Add Fetch Effective Address Time

†Add Calculate Effective Address Time

8.2.12 Shift/Rotate Instructions

The shift/rotate instructions table indicates the number of clock periods needed for the processor to perform the specified operation on the given addressing mode. Footnotes indicate when it is necessary to add another table entry to calculate the total effective execution time for the instruction. The number of bits shifted does not affect execution time. The total number of clock cycles is outside the parentheses, the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction		Best Case	Cache Case	Worst Case
LSL	Dn (Static)	1(0/0/0)	4(0/0/0)	4(0/1/0)
LSR	Dn (Static)	1(0/0/0)	4(0/0/0)	4(0/1/0)
LSL	Dn (Dynamic)	3(0/0/0)	6(0/0/0)	6(0/1/0)
LSR	Dn (Dynamic)	3(0/0/0)	6(0/0/0)	6(0/1/0)
*	LSL	Mem by 1	5(0/0/1)	6(0/1/1)
*	LSR	Mem by 1	5(0/0/1)	6(0/1/1)
	ASL	Dn	5(0/0/0)	8(0/0/0)
	ASR	Dn	3(0/0/0)	6(0/1/0)
*	ASL	Mem by 1	6(0/0/1)	7(0/1/1)
*	ASR	Mem by 1	5(0/0/1)	6(0/1/1)
	ROL	Dn	5(0/0/0)	8(0/1/0)
	ROR	Dn	5(0/0/0)	8(0/1/0)
*	ROL	Mem by 1	7(0/0/1)	7(0/1/1)
*	ROR	Mem by 1	7(0/0/1)	7(0/1/1)
	ROXL	Dn	9(0/0/0)	12(0/1/0)
	ROXR	Dn	9(0/0/0)	12(0/1/0)
*	ROXd	Mem by 1	5(0/0/1)	6(0/1/1)

*Add Fetch Effective Address Time

d—Direction of Shift/Rotate, L or R

8.2.13 Bit Manipulation Instructions

The bit manipulation instructions table indicates the number of clock periods needed for the processor to perform the specified bit operation on the given addressing mode. Footnotes indicate when it is necessary to add another table entry to calculate the total effective execution time for the instruction. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction			Best Case	Cache Case	Worst Case
	BTST	#<data>,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
	BTST	Dn,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
**	BTST	#<data>,Mem	4(0/0/0)	4(0/0/0)	5(0/1/0)
*	BTST	Dn,Mem	4(0/0/0)	4(0/0/0)	5(0/1/0)
	BCHG	#<data>,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
	BCHG	Dn,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
**	BCHG	#<data>,Mem	4(0/0/1)	4(0/0/1)	5(0/1/1)
*	BCHG	Dn,Mem	4(0/0/1)	4(0/0/1)	5(0/1/1)
	BCLR	#<data>,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
	BCLR	Dn,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
**	BCLR	#<data>,Mem	4(0/0/1)	4(0/0/1)	5(0/1/1)
*	BCLR	Dn,Mem	4(0/0/1)	4(0/0/1)	5(0/1/1)
	BSET	#<data>,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
	BSET	Dn,Dn	1(0/0/0)	4(0/0/0)	5(0/1/0)
**	BSET	#<data>,Mem	4(0/0/1)	4(0/0/1)	5(0/1/1)
*	BSET	Dn,Mem	4(0/0/1)	4(0/0/1)	5(0/1/1)

*Add Fetch Effective Address Time

** Add Fetch Immediate Address Time

8.2.14 Bit Field Manipulation Instructions

The bit field manipulation instructions table indicates the number of clock periods needed for the processor to perform the specified bit field operation using the given addressing mode. Footnotes indicate when it is necessary to add another table entry to calculate the total effective execution time for the instruction. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction			Best Case	Cache Case	Worst Case
	BFTST	Dn	3(0/0/0)	6(0/0/0)	7(0/1/0)
‡	BFTST	Mem (< 5 Bytes)	11(1/0/0)	11(1/0/0)	12(1/1/0)
‡	BFTST	Mem (5 Bytes)	15(2/0/0)	15(2/0/0)	16(2/1/0)
	BFCHG	Dn	9(0/0/0)	12(0/0/0)	12(0/1/0)
‡	BFCHG	Mem (< 5 Bytes)	16(1/0/1)	16(1/0/1)	16(1/1/1)
‡	BFCHG	Mem (5 Bytes)	24(2/0/2)	24(2/0/2)	24(2/1/2)
	BFCLR	Dn	9(0/0/0)	12(0/0/0)	12(0/1/0)
‡	BFCLR	Mem (< 5 Bytes)	16(1/0/1)	16(1/0/1)	16(1/1/1)
‡	BFCLR	Mem (5 Bytes)	24(2/0/2)	24(2/0/2)	24(2/1/2)
	BFSET	Dn	9(0/0/0)	12(0/0/0)	12(0/1/0)
‡	BFSET	Mem (< 5 Bytes)	16(1/0/1)	16(1/0/1)	16(1/1/1)
‡	BFSET	Mem (5 Bytes)	24(2/0/2)	24(2/0/2)	24(2/1/2)
	BFEXTS	Dn	5(0/0/0)	8(0/0/0)	8(0/1/0)
‡	BFEXTS	Mem (< 5 Bytes)	13(1/0/0)	13(1/0/0)	13(1/1/0)
‡	BFEXTS	Mem (5 Bytes)	18(2/0/0)	18(2/0/0)	18(2/1/0)
	BFEXTU	Dn	5(0/0/0)	8(0/0/0)	8(0/1/0)
‡	BFEXTU	Mem (< 5 Bytes)	13(1/0/0)	13(1/0/0)	13(1/1/0)
‡	BFEXTU	Mem (5 Bytes)	18(2/0/0)	18(2/0/0)	18(2/1/0)
	BFINS	Dn	7(0/0/0)	10(0/0/0)	10(0/1/0)
‡	BFINS	Mem (< 5 Bytes)	14(1/0/1)	14(1/0/1)	15(1/1/1)
‡	BFINS	Mem (5 Bytes)	20(2/0/2)	20(2/0/2)	21(2/1/2)
	BFFFO	Dn	15(0/0/0)	18(0/0/0)	18(0/1/0)
‡	BFFFO	Mem (< 5 Bytes)	24(1/0/0)	24(1/0/0)	24(1/1/0)
‡	BFFFO	Mem (5 Bytes)	32(2/0/0)	32(2/0/0)	32(2/1/0)

‡Add Calculate Immediate Address Time

NOTE: A bit field of 32 bits may span five bytes that require two operand cycles to access or may span four bytes that require only one operand cycle to access.

8.2.15 Conditional Branch Instructions

The conditional branch instructions table indicates the number of clock periods needed for the processor to perform the specified branch on the given branch size, with complete execution times given. No additional tables are needed to calculate total effective execution time for these instructions. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction	Best Case	Cache Case	Worst Case
Bcc (Taken)	3(0/0/0)	6(0/0/0)	9(0/2/0)
Bcc.B (Not Taken)	1(0/0/0)	4(0/0/0)	5(0/1/0)
Bcc.W (Not Taken)	3(0/0/0)	6(0/0/0)	7(0/1/0)
Bcc.L (Not Taken)	3(0/0/0)	6(0/0/0)	9(0/2/0)
DBcc (cc = False, Count Not Expired)	3(0/0/0)	6(0/0/0)	9(0/2/0)
DBcc (cc = False, Count Expired)	7(0/0/0)	10(0/0/0)	10(0/3/0)
DBcc (cc = True)	3(0/0/0)	6(0/0/0)	7(0/1/0)

8.2.16 Control Instructions

The control instructions table indicates the number of clock periods needed for the processor to perform the specified operation. Footnotes specify when it is necessary to add an entry from another table to calculate the total effective execution time for the given instruction. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction	Best Case	Cache Case	Worst Case
ANDI to SR	9(0/0/0)	12(0/0/0)	15(0/2/0)
EORI to SR	9(0/0/0)	12(0/0/0)	15(0/2/0)
ORI to SR	9(0/0/0)	12(0/0/0)	15(0/2/0)
ANDI to CCR	9(0/0/0)	12(0/0/0)	15(0/2/0)
EORI to CCR	9(0/0/0)	12(0/0/0)	15(0/2/0)
ORI to CCR	9(0/0/0)	12(0/0/0)	15(0/2/0)
BSR	5(0/0/1)	7(0/0/1)	13(0/2/1)
** CALLM (Type 0)	28(2/0/6)	30(2/0/6)	36(2/2/6)
** CALLM (Type 1)—No Stack Copy	48(5/0/8)	50(5/0/8)	56(5/2/8)
** CALLM (Type 1)—No Stack Copy	55(6/0/8)	57(6/0/8)	64(6/2/8)
** CALLM (Type 1)—Stack Copy	63 + 6n (7 + n/0/8 + n)	65 + 6n (7 + n/0/8 + n)	71 + 6n (7 + n/2/8 + n)
‡ CAS (Successful Compare)	15(1/0/1)	15(1/0/1)	16(1/1/1)
‡ CAS (Unsuccessful Compare)	12(1/0/0)	12(1/0/0)	13(1/1/0)
CAS2 (Successful Compare)	23(2/0/2)	25(2/0/2)	28(2/2/2)
CAS2 (Unsuccessful Compare)	19(2/0/0)	22(2/0/0)	25(2/2/0)
* CHK	8(0/0/0)	8(0/0/0)	8(0/1/0)
** CHK2 EA,Rn	16(2/0/0)	18(2/0/0)	18(2/1/0)
% JMP	1(0/0/0)	4(0/0/0)	7(0/2/0)
% JSR	3(0/0/1)	5(0/0/1)	11(0/2/1)
† LEA	2(0/0/0)	2(0/0/0)	3(0/1/0)
LINK.W	3(0/0/1)	5(0/0/1)	7(0/1/1)
LINK.L	4(0/0/1)	6(0/0/1)	10(0/2/1)
NOP	2(0/0/0)	2(0/0/0)	3(0/1/0)
† PEA	3(0/0/1)	5(0/0/1)	6(0/1/1)
RTD	9(1/0/0)	10(1/0/0)	12(1/2/0)
RTM (Type 0)	18(4/0/0)	19(4/0/0)	22(4/2/0)
RTM (Type 1)	31(6/0/1)	32(6/0/1)	35(6/2/1)
RTR	13(2/0/0)	14(2/0/0)	15(2/2/0)
RTS	9(1/0/0)	10(1/0/0)	12(1/2/0)
UNLK	5(1/0/0)	6(1/0/0)	7(1/1/0)

n—Number of Operand Transfers Required

*Add Fetch Effective Address Time

†Add Calculate Effective Address Time

%—Add Jump Effective Address Time

** Add Fetch Immediate Address Time

‡Add Calculate Immediate Address Time

8.2.17 Exception-Related Instructions

The exception-related instructions table indicates the number of clock periods needed for the processor to perform the specified exception-related action. Footnotes specify when it is necessary to add the entry from another table to calculate the total effective execution time for the given instruction. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Instruction	Best Case	Cache Case	Worst Case
BKPT	9(1/0/0)	10(1/0/0)	10(1/0/0)
Interrupt (I-Stack)	26(2/0/4)	26(2/0/4)	33(2/2/4)
Interrupt (M-Stack)	41(2/0/8)	41(2/0/8)	48(2/2/8)
RESET Instruction	518(0/0/0)	518(0/0/0)	519(0/1/0)
STOP	8(0/0/0)	8(0/0/0)	8(0/0/0)
Trace	25(1/0/5)	25(1/0/5)	32(1/2/5)
TRAP #n	20(1/0/4)	20(1/0/4)	27(1/2/4)
Illegal Instruction	20(1/0/4)	20(1/0/4)	27(1/2/4)
A-Line Trap	20(1/0/4)	20(1/0/4)	27(1/2/4)
F-Line Trap	20(1/0/4)	20(1/0/4)	27(1/2/4)
Privilege Violation	20(1/0/4)	20(1/0/4)	27(1/2/4)
TRAPcc (Trap)	23(1/0/5)	25(1/0/5)	32(1/2/5)
TRAPcc (No Trap)	1(0/0/0)	4(0/0/0)	5(0/1/0)
TRAPcc.W (Trap)	23(1/0/5)	25(1/0/5)	33(1/3/5)
TRAPcc.W (No Trap)	3(0/0/0)	6(0/0/0)	7(0/1/0)
TRAPcc.L (Trap)	23(1/0/5)	25(1/0/5)	33(1/3/5)
TRAPcc.L (No Trap)	5(0/0/0)	8(0/0/0)	10(0/2/0)
TRAPV (Trap)	23(1/0/5)	25(1/0/5)	32(1/2/5)
TRAPV (No Trap)	1(0/0/0)	4(0/0/0)	5(0/1/0)

8.2.18 Save and Restore Operations

The save and restore operations table indicates the number of clock periods needed for the processor to perform the specified state save or return from exception, with complete execution times and stack length given. No additional tables are needed to calculate total effective execution time for these operations. The total number of clock cycles is outside the parentheses; the number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). These cycles are included in the total clock cycle number.

Operation	Best Case	Cache Case	Worst Case
Bus Cycle Fault (Short)	42 (1/0/10)	43 (1/0/10)	50 (1/2/10)
Bus Cycle Fault (Long)	79 (1/0/24)	79 (1/0/24)	86 (1/2/24)
RTE (Normal)	20 (4/0/0)	21 (4/0/0)	24 (4/2/0)
RTE (Six Word)	20 (4/0/0)	21 (4/0/0)	24 (4/2/0)
RTE (Throwaway)*	15 (4/0/0)	16 (4/0/0)	39 (4/0/0)
RTE (Coprocessor)	31 (7/0/0)	32 (7/0/0)	33 (7/1/0)
RTE (Short Fault)	42 (10/0/0)	43 (10/0/0)	45 (10/2/0)
RTE (Long Fault)	91 (24/0/0)	92 (24/0/0)	94 (24/2/0)

*Add the time for RTE on second stack frame.